UM10060

毫米波传感器开发工具用户手册

Rev.1.4

用户手册

介绍

本文档旨在阐述矽典微 ONELAB 毫米波传感器开发板毫米波传感器开发工具的使用方法，并通过详细解析频率、天线配置、波形类型及数据输出格式等关键指标，为用户提供一种便捷且直观的方式来设置毫米波传感器参数。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

目 录

1 环境准备 - 3 -
2 主窗口界面说明 - 4 -
2.1 菜单 - 4 -
2.2 设备操作按钮区 - 6 -
2.3 回放信息区 - 7 -
2.4 波形显示配置区 - 7 -
2.5 配置信息展示区 - 7 -
2.6 目标信息展示区 - 7 -
2.7 波形/日志显示窗口 - 7 -
2.8 热图信息区 - 8 -
3 寄存器配置界面 - 8 -
3.1 主界面介绍 - 8 -
3.2 参数配置页面 - 20 -
3.3 常用参数说明 - 22 -
4 数据格式 - 24 -
4.1 芯片数据格式 - 24 -
4.2 MCU 数据封装 - 29 -
5 通信协议 - 29 -
5.1 协议概述 - 29 -
5.2 帧结构 - 29 -
5.3 帧数据发送顺序 - 30 -
5.4 帧内数据 - 30 -
5.5 通信流程 - 30 -
5.6 通信示例 - 32 -
6 毫米波传感器开发工具操作示例 - 33 -
7 常见问题及解决方案 - 36 -
7.1 无法连接设备 - 36 -
7.2 配置后上位机不输出数据 - 37 -
7.3 如何计算实际输出的数据 - 41 -
7.4 如何配置 Chirp 时间及采样信息 - 43 -
8 自定义硬件开发 - 45 -
8.1 硬件设计 - 45 -
8.2 MCU 选型 - 45 -
8.3 固件移植 - 47 -
8.4 波形参数配置 - 58 -
9 版本历史 - 60 -
重要声明 - 61 -

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

1 环境准备

- **操作系统**：Windows 10-64 bit
- **硬件环境**：
  - 已烧录数据透传固件的毫米波传感器开发板(DEV 系列模组)；
  - 串口转接板：部分模组使用 3.3V 电源供电，需要保证转接板支持的电流能满足模组所需峰值电流。

下表为 DEV 系列 3.3V 电源供电模组的峰值电流典型值:

表 1-1 DEV 系列 3.3V 电源供电模组的峰值电流典型值

毫米波传感器名称	输入电压（V）	峰值电流典型值（mA）
DEV101	3.3	97
DEV101Pro	3.3	115
DEV102	5	82
DEV103	3.3	98
DEV103Pro	3.3	115
DEV101A(Xen101S)	3.3	115
DEV101Apro（EVB111A_S)	3.3	140
DEV103A	3.3	130
DEV103Apro（EVB111A_L)	3.3	140
DEV202H	5	118
DEV202S	3.3	150
DEV202E	5	115
DEV202V	3.3	140
DEV111E	5	150
DEV111V	3.3	175
DEV111P	3.3	165
DEV112P	5	202
DEV113P	3.3	310

- **软件环境**：
  - VC 运行库：

    64 位（x64）：https://aka.ms/vs/17/release/vc_redist.x64.exe

    32 位（x86）：https://aka.ms/vs/17/release/vc_redist.x86.exe
- **授权码**：可通过官网获取

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

2 主窗口界面说明

本章介绍毫米波传感器开发板开发工具的主窗口界面各模块的外观与功能，界面布局如图 2-1 所示。界面区域划分：

- 设备操作按钮区
- 回放信息区
- 波形配置区
- 设备配置展示区
- 目标信息展示区
- 波形/日志显示窗口
- 热图信息区

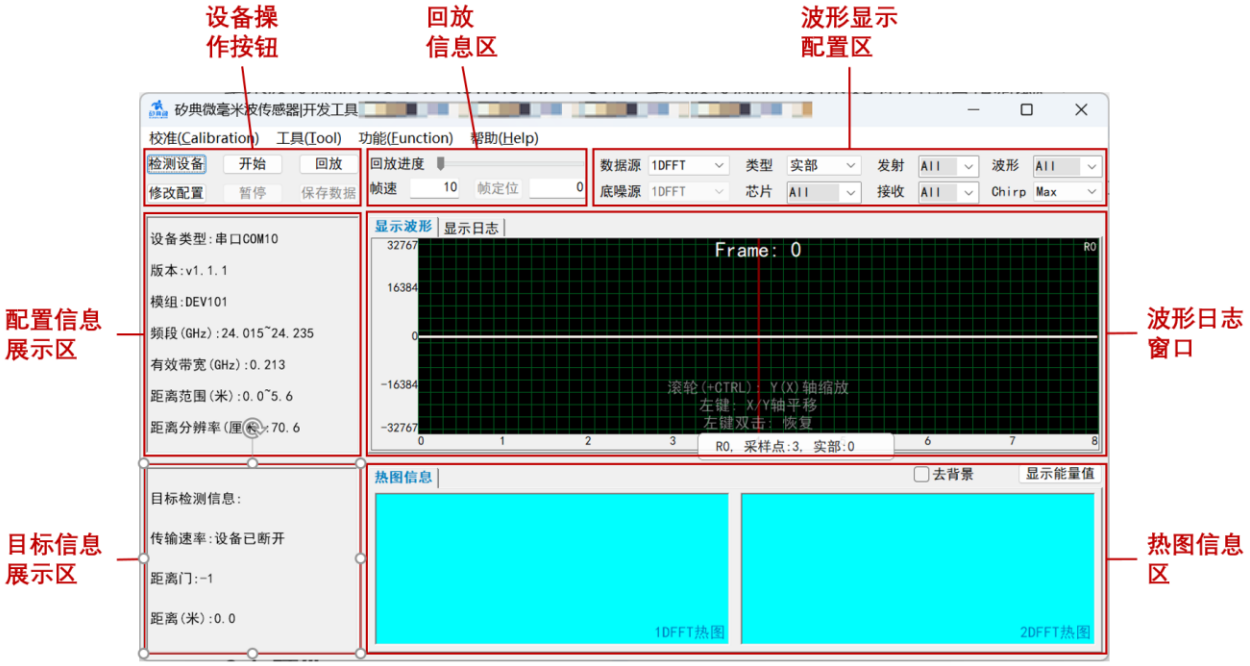


图 2-1 毫米波传感器开发板开发工具主窗口界面

2.1 菜单

- **“校准”**：提供芯片相关的校准功能，包括：

  IVCM 校准：在不改变档位功率的情况下，调整 IVCM 寄存器，使 ADC 幅值的波峰波谷尽可能居中对称。

  注意：IVCM 校准需要使用 RAW 数据进行校准，校准时需要配置芯片输出格式为 DSRAW。

  ADC 权重校准（仅 ICL1240 模组支持）：ADC 校准权重，用以改善 ADC 性能。

  LVDS 校准（仅 ICL1240 模组支持）：调整 LVDS bclk/fclk/data 之间的相位关系，以保证 LVDS 数据输出的稳定性。

- **“工具”**：提供基于模组的工具操作，包括：

  启动演示程序：根据模组名称，启动对应的演示工具。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

模组固件更新：OTA 升级功能，针对模组的固件升级。

芯片固件更新（仅 ICL1240 模组支持）：升级 ICL1240 模组内 FLASH 中的 APP，模组内需预先烧录 FSBL 相关信息。

- **“功能”**：附加功能，包括：

温度上报间隔：此功能用于开启/关闭实时温度显示功能，需要固件支持温度上报功能。开启此功能后，会默认开启“MCU 数据封装”，数据格式需按照“MCU 数据封装”格式解析。温度需要校准后才能与实际温度更匹配。工具中显示的温度未经过校准，如需校准，请参考《芯片温度检测模块使用说明》。

注意：如果固件未支持此功能(DEV 系列 1.0.2 以前的版本均不支持)，可能造成数据格式解析错误，如发生此情况，请关闭温度上报功能。

保存隔离度：此功能用于保存当前芯片数据的隔离度信息，报表生成于程序目录下的 PAdc 目录下。

注意：必须是 DSRAW 类型的数据格式，才可以收集隔离度信息。

IDAC 设置（仅 ICL1240 模组支持）：配置 IDAC 波形参数。IDAC 是内部可配置的基带注入信号，可以注入各种特殊的波形。

数据导出设置：可启用数据导出功能。启用后，将在程序目录下生成 ObjectInfo 目录。每次启动毫米波传感器后，采集指定时间间隔内的芯片数据统计信息，包括原始数据，能量值/模值等信息。

功率档位测试：此功能用于遍历所有 Tx/Rx 档位，并采集对应档位的芯片数据。

重复启动测试：此功能用于重复开始/停止操作，用于毫米波传感器稳定性测试。

- **“帮助”**：用户手册，授权等信息，包括：

切换语言：可选择中文/英文界面，切换语言需重新启动软件后生效。

用户手册：点击后打开用户手册（本文档）。

检查更新：点击后立刻检查新版本信息(默认每 30 秒检查一次)。

授权信息：点击后查看当前授权信息，以及授权信息更新功能。

本软件使用时需要匹配对应的授权文件，授权文件可通过官网获取。

未获取授权或授权过期时，程序启动后会弹出如下授权窗口。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。



图 2-2 未授权或授权过期提醒界面

点击“更新授权信息”窗口，填入对应设备 ID 的授权文件(.lic)：



图 2-3 “更新授权”界面

授权成功后，授权信息窗口更新如下：

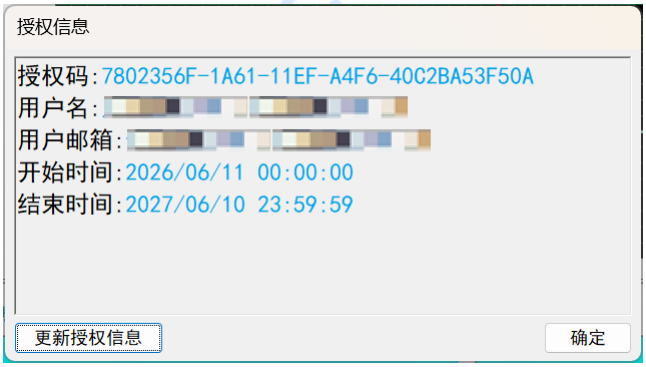


图 2-4 授权信息界面

后期也可以通过菜单“帮助->授权信息”更新授权文件。

关于：点击后查看“终端用于许可说明 --- EULA”。

2.2 设备操作按钮区

- **“检测设备”**：点击后加载程序运行目录下的 Radar_Config.ini 配置文件，并据此配置检测对应硬件信息。

  注意：对于一个新的模组，需要重新生成对应模组的配置。模组的配置生成请参考：配置操作。

- **“修改配置”**：调整传感器配置信息和上位机处理参数。
- **“开始/停止”**：启动/停止数据传输，支持子板数据传输和文件数据回放两种模式。
- **“暂停/继续”**：暂停/恢复数据传输。子板模式下，“暂停”不中断传输但丢弃数据；回放模式下，“暂停”停止

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

读取但不丢弃数据。

- **“回放”**：选择数据目录后，点击“开始”回放文件数据。
- **“保存数据”**：启动数据保存功能，将检测数据保存在指定目录下。

2.3 回放信息区

- **回放进度条**：实时显示回放进度。
- **帧速输入框**：设置回放帧速度，当帧速 < 0 时，开启逐帧回放模式，用户可手动点击“继续”按钮进行逐帧回放。
- **帧定位输入框**：直接定位至指定数据帧。

2.4 波形显示配置区

- **数据源**：可选“RAW”、“1DFFT”或“2DFFT”。
- **底噪源**：用于计算底噪，可选“1DFFT”或“2DFFT”。底噪计算方法为：先统计每个距离门上能量的中位均值（所有 Chirp 按对应距离门排序后的中间 1/3 数据量），再统计所有距离门上的能量的中位均值（1/3 距离门数量）。
- **类型**：当数据类型为 RAW 时，可选择“幅值”或“隔离度”；当数据源为 1DFFT/2DFFT 时可选“实部”“虚部”“相位”“模值”“能量”“信噪比”。
- **芯片/发射/接收**：分别显示指定芯片、发射/接收通道波形。
  - **RAW**：选择对应的 RX 的 I(偶数)/ Q(奇数) 路进行显示
  - **1DFFT /2DFFT**：选择对应的 RX 进行显示。
- **波形**：选择查看的波形序号。
- **Chirp**：选择查看特定 index 的 chirp，默认“Max（幅值绝对值最大的 chirp）”。多波形、降采样或多芯片交替发射时，chirp 有效数量=chirp 数量/(波形数量*降采样*发射数量)。

2.5 配置信息展示区

显示当前传感器的配置文件信息。

2.6 目标信息展示区

显示检测到目标的距离、速度、信噪比等信息。

2.7 波形/日志显示窗口

- **显示波形**：展示接收波形，纵坐标为数值（ADC 值、1D/2DFFT 实/虚部等），横坐标为数据位置（RAW 为

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

采样点，1D/2DFFT 为距离门）。鼠标悬停在该区域时会显示悬停点的数值，可平移、缩放波形。

- **平移**：左键按下拖动。
- **恢复默认**：左键双击。
- **纵坐标缩放**：滚轮。
- **横坐标缩放**：Ctrl +滚轮。

- **显示日志**：显示日志信息，非显示状态时后台保存最后 20000 条。

2.8 热图信息区

- **1DFFT 热图**：显示 1DFFT 能量，横坐标为距离门，纵坐标为 Chirp 数，按 index 排序。根据波形配置提取最大值显示。
- **2DFFT 热图**：显示 DPL 热图，横坐标为距离门，纵向表示速度维（中点速度 0，向上速度增，向下速度减）。根据波形配置提取最大值显示。
- **“去背景”勾选框**：点击后显示去背景噪声后的热图。
- **“显示能量值”按钮**：点击后弹出能量值对话框。

3 寄存器配置界面

3.1 主界面介绍

寄存器配置主界面划分为以下区域，具体布局如图 3-1 所示：

- 芯片输出配置
- 波形设置
- 数据输出设置
- 固件设置
- 波形信息展示
- 波形示意图
- 寄存器信息
- 配置操作

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

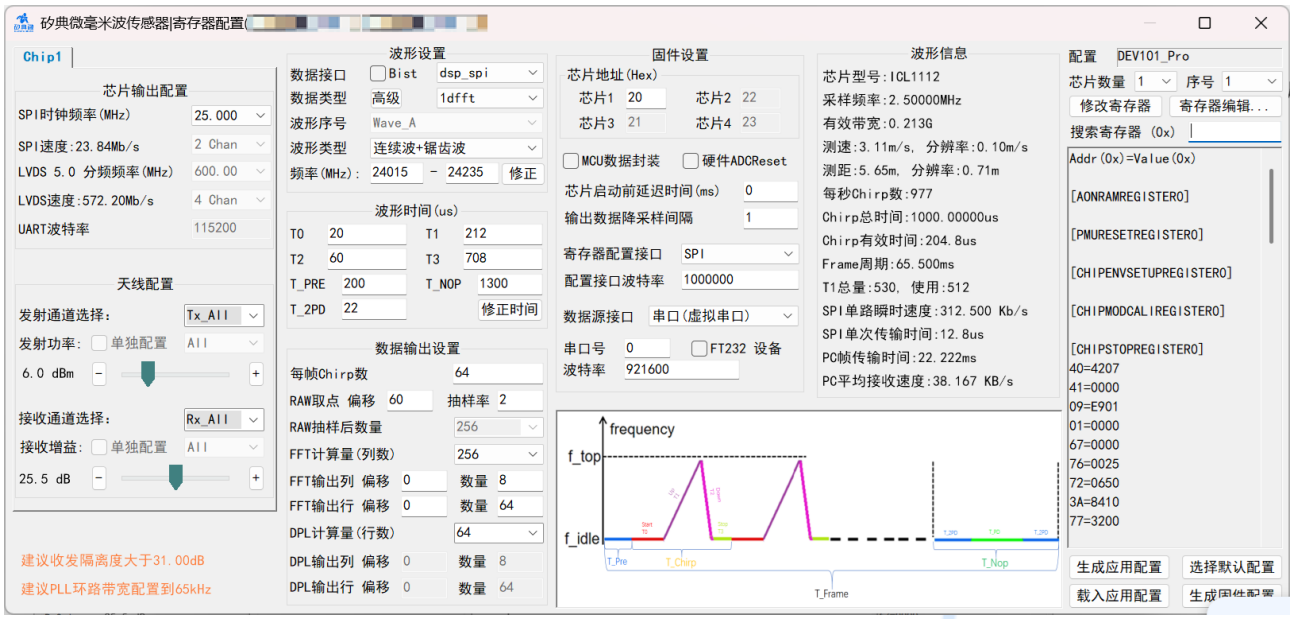


图 3-1 寄存器配置界面

3.1.1 芯片输出配置

注意：该配置页面中所有配置仅针对当前选中芯片。

3.1.1.1 芯片输出配置



图 3-2 芯片输出配置图片

- **SPI 时钟频率（MHz）**：设定芯片 SPI 时钟。
- **SPI 速度**：由 SPI 时钟频率转换得出的传输速度（1000 转为 1024）。
- **SPI 通道数**：芯片输出时使用的通道数。如果选择 1，则表示芯片将采用 Merge 方式将数据打包输出到一路 SPI 通道上。
- **LVDS 分频频率（MHz）**：支持 5、6、7、7.5、8、15 分频。
- **LVDS 速度 & 通道数**：LVDS 速度由时钟频率转换得出的传输速度（1000 转为 1024）。LVDS 通道支持 1 路、2 路、4 路三种模式。
- **UART 波特率**：数据输出为 UART 时，设定芯片 UART 输出速度。

3.1.1.2 天线配置

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

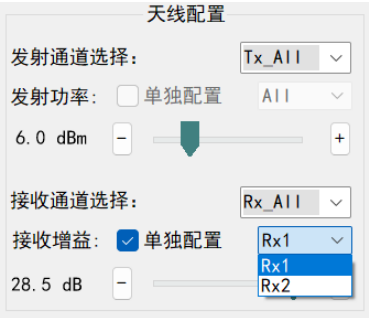


图 3-3 天线配置图

- **发射通道选择**：选择发射通道（默认全选）。
- **发射功率/EIRP**： 天线发射功率/EIRP 值；当发射天线数量>1 时，可勾选”单独配置”对每个发射通道单独设置功率档位。
- **接收通道选择**：选择接收通道（默认全选）。
- **接收增益**：天线接收增益值；当接收天线数量>1 时，可勾选”单独配置”对每个接收通道单独设置增益档位。

3.1.2 波形配置

3.1.2.1 波形设置

- **数据接口**：选择芯片数据源（SPI 或 UART）。
- **数据类型**：设定输出数据格式，如 RAW、1DFFT、2DFFT 等。
- **波形序号（仅 ICL1240 模组支持）**：选择查看对应波形的参数设置，以及多波形配置。当选定某个波形时，可修改对应波形的参数配置。切换芯片数据格式时，会统一波形参数。

  注意：波形参数可支持如下差异化配置：

  发射功率(仅当 Tx Ldo 源为 Patgen 时生效)

  波形频率

  波形时间

  波形输出

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

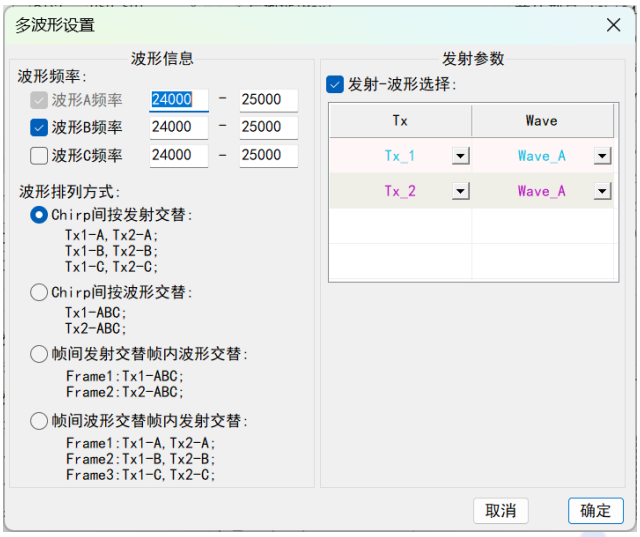


图 3-4 ICL 系列高级配置界面示例

- **波形频率**：可设置使能 B 波或 C 波，并设置对应的波形频率。
- **波形排列方式：**
  - ✓ Chirp 间按发射交替：波形在 Chirp 间按照 Tx1-A, Tx2-A, Tx1-B, Tx2-B, Tx1-C, Tx2-C 的顺序循环
  - ✓ Chirp 间按波形交替：波形在 Chirp 间按照 Tx1-A, Tx1-B, Tx1-C, Tx2-A, Tx2-B, Tx2-C 的顺序循环
  - ✓ 帧间发射交替帧内波形交替：第一个 Frame 内，波形在 Chirp 间按照 Tx1-A, Tx1-B, Tx1-C 的顺序循环；第二个 Frame 内，波形在 Chirp 间按照 Tx2-A, Tx2-B, Tx2-C 的顺序循环；依次重复
  - ✓ 帧间波形交替帧内发射交替：第一个 Frame 内，波形在 Chirp 间按照 Tx1-A, Tx2-A 的顺序循环；第二个 Frame 内，波形在 Chirp 间按照 Tx1-A, Tx2-B 的顺序循环；第三个 Frame 内，波形在 Chirp 间按照 Tx1-C, Tx2-C 的顺序循环；依次重复
- **发射波形选择：**DSP 模块在多波形数据处理或输出时设置 Tx 与波形的对应关系。主要用于 IIR，DFFT 处理等。

- **波形类型：**设定芯片调制信号类型，支持“连续波模式”、“连续波+锯齿波”、“连续波+三角波”三种波形模式。

  连续波模式：雷达发射机持续发射**固定频率**的连续射频载波，无频率 / 幅度的调制变化，接收机接收目标反射的回波信号，通过**多普勒频移**实现目标测速的基础波形模式。此模式下，Chirp 在 T0~T3 时间段内均发射固定频率信号。

  连续波+锯齿波：锯齿波属于**线性调频连续波（LFMCW）** 基础模式，雷达发射机在连续波基础上，使载波频率**随时间线性上升**（单斜率扫频），达到预设最大频率后**跳回初始频率**。选择此模式时，Chirp 在 T0 和 T3 发射固定频率信号，在 T1 和 T2 时间段内发射锯齿波信号。

  连续波+三角波：三角波指载波频率**随时间先线性上升、再线性下降**（双斜率扫频），上升沿和下降

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

沿的扫频斜率大小相等、方向相反，形成 “三角状” 的 f-t 波形。选择此模式时，Chirp 在 T0 和 T3 发射固定频率信号，在 T1 和 T2 时间段内发射三角波信号。

- **频率（MHz）**：设定扫频范围；“波形类型”为“连续波”时，仅可设置起始频率。
  - **修正[1]**：根据精度和 chirp 时间，计算实际的频率范围。

3.1.2.2 高级设置

“高级”设置页面用于配置除芯片基础配置外的其他关键参数。

- **S5KM312CL 系列高级设置**



图 3-5 S5KM312CL 系列高级配置界面示例

- **低功耗级别**：选择是否开启低功耗。
- **FFT 窗函数**：选择 FFT（1D/2D）窗函数。
- **DCDC 开关频率**：控制 DCDC 开关频率，单位 KHz。

- **ICL111A/ICL1112/ICL1122 系列高级设置**



图 3-6 ICL111A/ICL1112/ICL1122 系列高级配置界面示例

- **低功耗级别**：选择是否开启低功耗。
- **ADC 饱和检测**：勾选后，可以置 ADC 饱和检测功能的精度，数值越大表示检测频次越高。

[1]每次点击“修正”按钮，工具都会根据界面上的参数重新计算实际生效频率，并将实际生效频率显示到界面中。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- **IIR 滤波器：**勾选后，使能 IIR 滤波功能，并可设置 IIR 滤波器系数；系数越高，对慢速目标的检测越敏感。
- **SPI Slave Mode：**勾选后，将芯片配置为从芯片模式。
- **DMA 阈值：**配置芯片输出数据时触发 data ready 信号的阈值。
- **DCDC 开关频率：**用于配置 DCDC 开关频率，单位 KHz（仅支持 ICL1112/ICL1122）。
- **CFAR 参数设置：**CFAR 即 Constant False Alarm Rate，恒虚警率；CFAR 系数越高，检测概率和虚警概率越低。
- **1DFFT 窗函数：**配置 1DFFT 运算时使用的窗函数。
- **1DFFT 窗起点/大小：**配置 1DFFT 运算的窗范围。
- **2DFFT 窗函数：**配置 2DFFT 运算时使用的窗函数。
- **2DFFT 窗起点/大小：**配置 2DFFT 运算的窗范围。
- **新数据格式：**勾选后，使用新数据格式（推荐）；否则，使用与 S5KM312CL 芯片兼容的数据格式。
- **寄存器一键启动：**勾选后，可一键启动寄存器配置，减少芯片开关时需要配置的寄存器数量。
- **帧头自动复位：**勾选后，每帧开始时自动重置 CIC3/CIC5/HB 滤波器。
- **Chirp 头自动复位：**勾选后，每个 chirp 开始时自动重置 CIC3/CIC5/HB 滤波器。

- **ICL1240 系列高级设置**

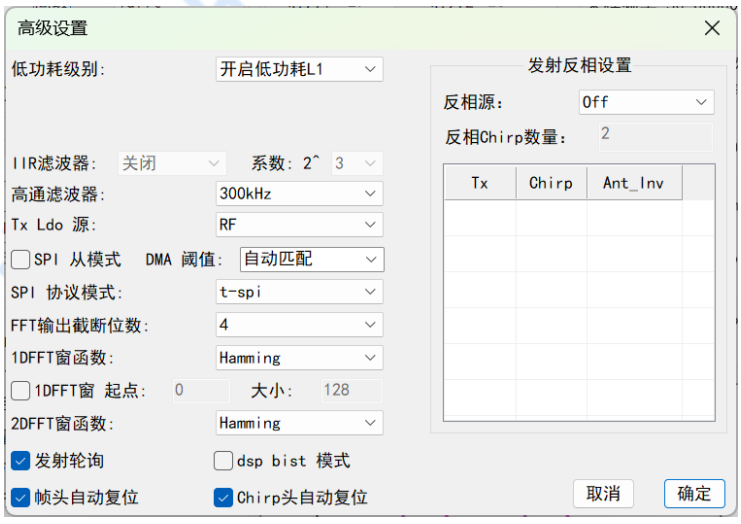


图 3-7 ICL1240 系列高级配置界面示例

- **低功耗级别：**选择是否开启低功耗。
- **IIR 滤波器：**使能 IIR 滤波功能，并可设置 IIR 滤波器系数；系数越高，对慢速目标的检测越敏感。（级联时，必须在多波形配置中开启发射波形选择后才能使能该功能）。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- **高通滤波器：**设置高通滤波器频率。
- **Tx Ldo 源：**设置发射功率配置方式。RF: 所有波形均使用统一的射频参数；Patgen: 每个波形可独立控制发射功率。
- **DMA 阈值：**配置芯片输出数据时触发 data ready 信号的阈值。
- **SPI 从模式：**勾选后，将芯片配置为从芯片模式。
- **SPI 协议模式：**可配置 SPI 工作模式，包含 T-SPI, D-SPI, Q-SPI 三种模式。
- **1DFFT 窗函数：**配置 1DFFT 运算时使用的窗函数。
- **1DFFT 窗起点/大小：**配置 1DFFT 运算的窗范围。
- **2DFFT 窗函数：**配置 2DFFT 运算时使用的窗函数。
- **2DFFT 窗起点/大小：**配置 2DFFT 运算的窗范围。
- **帧头自动复位：**勾选后，每帧开始时自动重置 CIC3。
- **Chirp 头自动复位：**勾选后，每个 chirp 开始时自动重置 CIC3。
- **反相：**用于设置在对应 Chirp 是否相位翻转的循环逻辑。
  - ✓ 反相源：支持开关反相设置，以及选择 RF（射频寄存器）/Patgen（Pll 寄存器）/TxMimo(数字寄存器)三种配置方法；
  - ✓ Chirp 数量：选择 RF 时，仅支持 1 个 Chirp 的循环配置；选择 Patgen 时，支持最多 12 个 chirp 的循环配置；选择 TxMimo 时，支持最多 32 个 chirp 的循环配置。

3.1.2.3 波形时间

波形时间由 **Chirp 时间**和**帧时间**组成；

Chirp 时间 由如下几段组成：

- **T0：**Chirp 启动时间。
- **T1：**Chirp FM 上升阶段时间，一般用于扫频模式下信号处理的数据采样。
- **T2：**Chirp FM 下降阶段时间（锯齿波时≥1/4 T1；三角波时=T1）。
- **T3：**Chirp 停止时间。

帧时间 由如下几段组成：

- **Chirp 总时间：Chirp 时间 * 每帧 Chirp 数。**
- **T_PRE 时间（us）：**帧开始到 Chirp 开始的时间。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- **T_NOP 时间（us）**：Chirp 结束到帧结束之间的时间。
- **T_2PD 时间**：勾选后，芯片在 T_NOP 期间进入 Power-Down 模式，设置 Power-Down 准备时间。
- **修正时间**：点击后按照最小采样时间修正界面中设置的时间

注意：配置波形时间时，需要满足以下两个条件：

- *条件 1[必须]：Chirp 采样的数据量必须要在 Chirp 时间内能输出。（即 数据量 / 芯片输出速率 <Chirp 时间）*
- *条件 2[应该]：当芯片数据需要通过 MCU 透传给其他设备（如 PC）时，需满足透传的传输速度 < 通信速率。（例如，每帧透传的数据量 ÷ 帧时间 < 通信速率）*

3.1.3 数据输出设置

- **每帧 Chirp 数**：设定每帧 Chirp 数量。
- **RAW 取点参数**：
  - 偏移：DSP 从 RAW 取数据的起始位置。
  - 抽样率：DSP 从 RAW 取数据的取点间隔。
- **RAW 抽样后数量**：
- **FFT 计算量（列数）**：设置 1DFFT 的计算量，即 FFT 使用的 RAW 抽样后点数。
- **FFT 输出列**：
  - 偏移：1DFFT 输出的列起始位置。
  - 数量：1DFFT 输出的列的个数。
- **FFT 输出行**：
  - 偏移：1DFFT 输出的行起始位置。
  - 数量：1DFFT 输出的行的个数。
- **DPL 计算量（行数）**：设置 2DFFT 的计算量。
- **DPL 输出列**：
  - 偏移：用于设置 2DFFT 输出列的起始位置。
  - 数量：用于设置 2DFFT 输出列的个数。
- **DPL 输出行**：
  - 偏移：用于设置 2DFFT 输出行的起始位置。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- 数量：用于设置 2DFFT 输出行的个数。

3.1.4 固件设置



图 3-8 固件设置界面

- **芯片地址（Hex）**：设定芯片设备地址。
- **MCU 数据封装**：固件在上传数据时，额外封装一层数据格式。常用于温度数据上报，或多个通道间的数据区分
- **芯片启动前延迟时间（ms）**：写入 start 段寄存器前的延迟时间，用于芯片稳定或等待 MCU 同步。
- **输出数据降采样间隔**：子板输出数据时降采样，设定输出帧内 Chirp 的间隔。例如：当每帧 Chirp 数设置为 32 时，输出 Chirp 间隔为 8，则子板输出的 Chirp 为：0，8，16，24，0，8，16，24…
- **寄存器配置接口**：配置 MCU 主板寄存器使用的接口。
- **配置接口波特率**：请参考模组手册，获取模组接口波特率
- **数据源接口**：子板数据输出接口，支持如下类型：
  - **串口（虚拟串口）**：串口连接板（或 USB 连接线），需设定串口号和波特率。对于 FT232 转接板，勾选 FT232 设备，可有效提升数据传输稳定性。



图 3-9 虚拟串口配置建议

  - **LibUSB**：支持 WINUSB/LibUSB 的驱动设备，需设定 USB 总线编号（BUS）、端口号（Port）、厂商 ID（VID）和设备 ID（PID），当任一参数值被指定为 <= 0 时，则不进行相应匹配。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

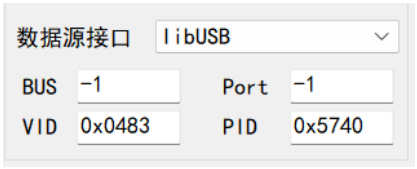


图 3-10 LibUSB 配置建议

用户可以在设备管理器中查看设备的 BUS(HUB)号和 PORT 号，如图 3-11 所示。

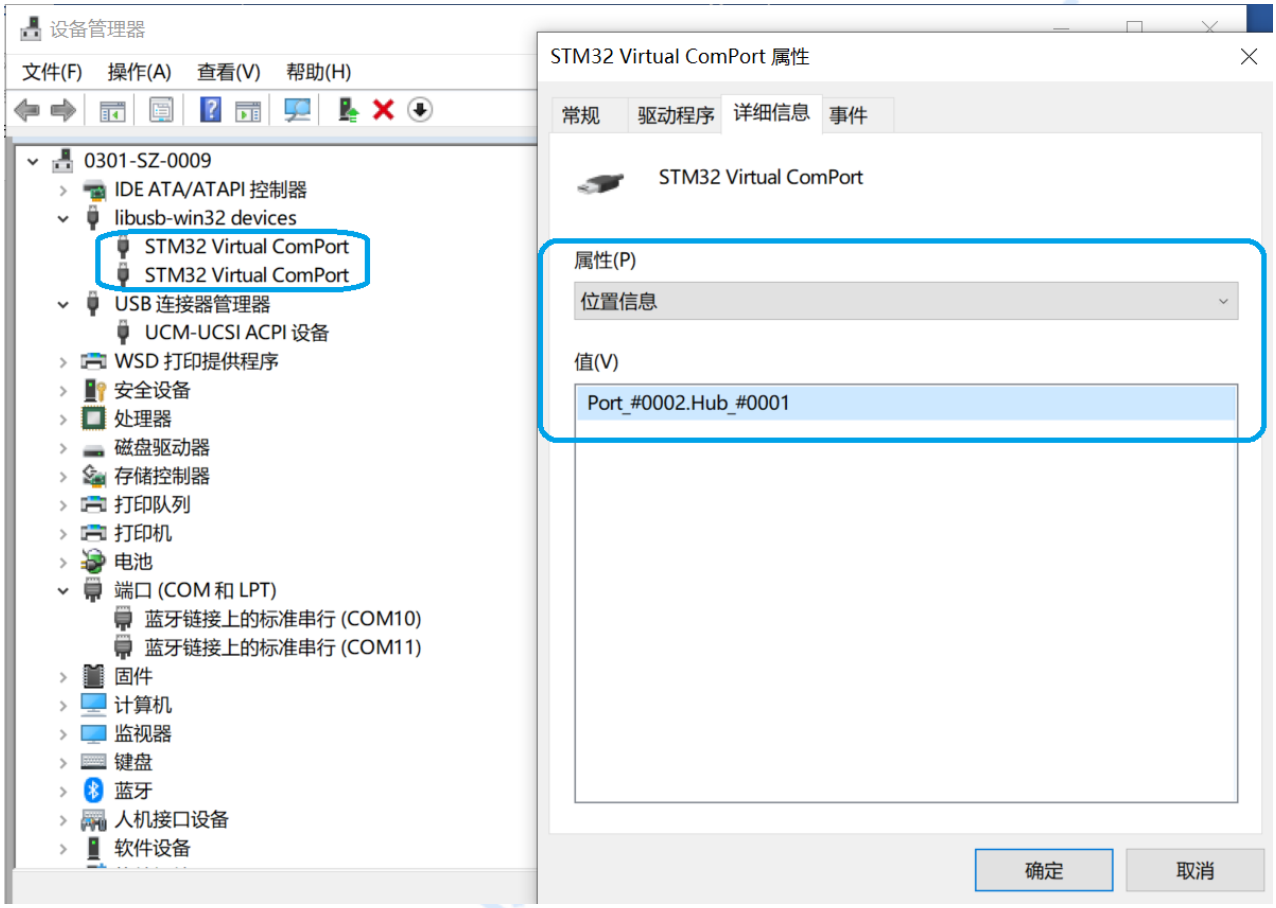


图 3-11 在设备管理器中查看设备端口号示例

- **DEVFB4**：FPGA 数据采集板，需设定 IP 地址和端口。该采集板使用固定 IP 地址：192.168.1.10，数据传输端口为 7，命令控制端口为 9。

3.1.5 波形信息展示

展示当前配置下的汇总信息包括：

芯片型号：当前配置使用的毫米波传感器芯片型号。

采样频率：传感器的 ADC 每秒采样次数。

有效带宽：实际使用到的带宽大小，仅扫频模式下有效。有效带宽=配置带宽 * T1 有效数据量 ÷ T1 总量。

测速：速度检测的最大范围，速度范围=速度分辨率 * 2DFFT 计算量 ÷ 2。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

速度分辨率：光速 ÷ 扫频中点 ÷ (2 * 2DFFT 计算量 * Chirp 时间)。

测距：距离检测的最大范围，一般等于 距离分辨率 * FFT 输出数量（列）。

距离分辨率：光速 ÷ (2 * 有效带宽) * T1 实际使用的 RAW 数据量 ÷ FFT 使用的数据量。

每秒 Chirp 数：每帧 Chirp 数量 * 每秒帧数；每秒帧数=1 秒 ÷ Frame 周期。

Chirp 总时间：T0+T1+T2+T3。

Chirp 有效时间：T1 有效数据量 ÷ 采样频率。

Frame 周期: Chirp 总时间 * 每帧 Chirp 数量 + T_PRE + T_NOP。

T1 总数据量：T1 时间 * 采样频率。

T1 有效数据量：RAW 抽样率 * RAW 抽样后数量。

SPI/UART 单路瞬时速度：单路 SPI/UART 上单个 Chirp 输出数据量 ÷ Chirp 时间；此值需要小于芯片输出速度，否则会导致芯片数据无法输出。

SPI/UART 单次传输时间：单路 SPI/UART 上单个 Chirp 输出数据量 ÷ SPI/UART 时钟频率。

PC 帧传输时间：帧数据量 ÷ PC 传输速率。

PC 平均接收速度：帧数据量 ÷ 帧时间。当设置帧时间接近 0 时，此时的平均接收速度可以代表 Chirp 级别的实时传输速度。

3.1.6 波形示意图

展示当前模式的 frame 波形示意图，便于使用者理解帧时间和 Chirp 时间。

3.1.7 寄存器信息

模组：子板模组信息。

芯片数量：不支持修改。

序号：可选择查看对应芯片的寄存器信息。

修改/保存寄存器：点击后寄存器文本框可编辑；编辑后点击“应用寄存器”保存。但若编辑后在界面中操作了其他参数配置（即使参数无变化），都会触发寄存器自动生成机制，导致修改后的寄存器被覆盖。编辑时，必须保留[TAG]信息，且[TAG]段用空行分隔。

搜索寄存器（0x）：输入 16 进制地址或值(不含 0x)，快速定位到寄存器文本框。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

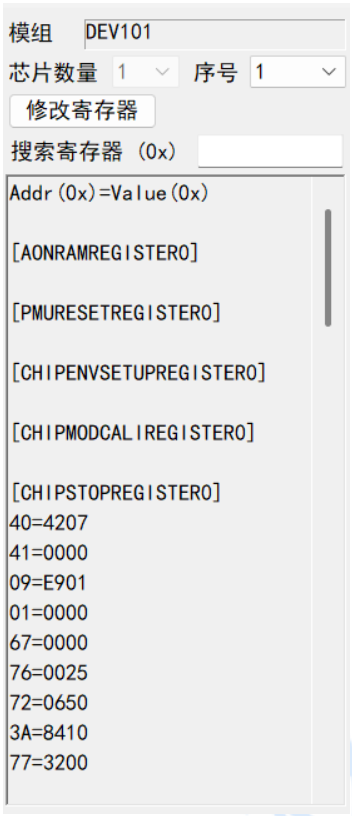


图 3-12 寄存器信息页面

3.1.8 配置操作

生成应用配置：点击后显示“参数配置”页面（页面详情见 3.2 节），用户可修改配置，并通过点击“保存”按钮保存当前寄存器配置到文件，供上位机软件使用。

选择默认配置：点击后弹出“加载默认配置”页面，用户可根据模组信息选择默认配置，点击“生成”按钮即可更新寄存器配置，并同步更新到“寄存器信息”页面。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。



图 3-13 “加载默认配置”页面

生成固件配置：保存当前寄存器配置到文件，供子板驱动使用。

载入应用配置：从已有应用配置文件中载入配置信息。

说明：默认配置中包含了一组芯片通用配置: S5KM312CL、ICL1122、ICL1112、ICL111A、ICL1240。

选择默认配置的时候，尽量选择和实际模组相同的配置(**ONELAB 硬件可以直接选择对应的模组配置**)。如果您的模组不在列表（非通用配置）中时，可以选择对应芯片的通用配置，进行手动配置。

选择通用配置后，一般需要根据您的实际模组进行如下修改：

a) 芯片数据接口：根据固件选择对应的数据接口（固件一般使用 SPI 接口）。

b) 芯片输出时钟配置：根据芯片输出的数据接口，设置对应 SPI/UART 的波特率。

c) 数据源接口：选择模组实际输出接口。

d) Chirp 时间配置：参考如何配置 Chirp 时间及采样信息。

3.2 参数配置页面

“参数配置”页面用于设置上位机工具的数据存储、目标检测以及协议交互相关参数。

在“寄存器配置”页面，点击“生成应用配置”按钮可弹出“参数配置”页面，如图 3-14 所示。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

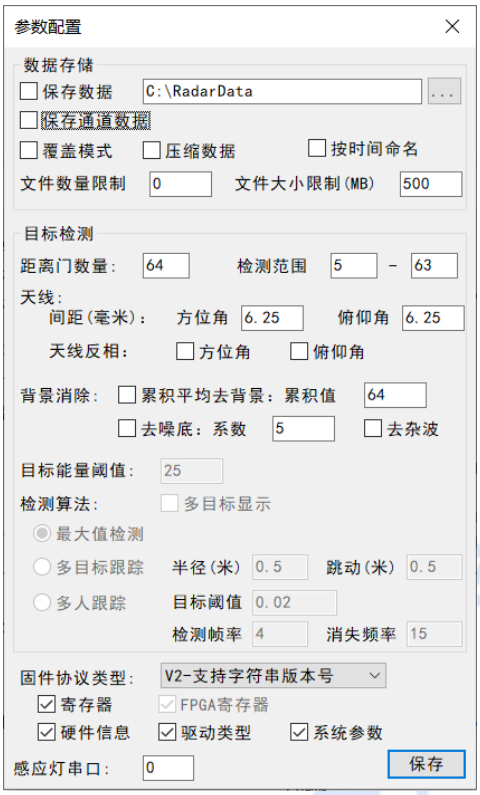


图 3-14 参数配置子界面示例

3.2.1 数据存储参数

保存数据：勾选后，系统将实时保存传感器数据。点击“...”按钮可选择存储目录。

保存通道数据：勾选后，保存数据时，程序额外按通道保存数据。

覆盖模式：不勾选时，每次保存数据将按日期时间生成新文件夹；勾选后，数据将固定存储于 RadarData_Update 目录。

压缩数据：勾选后，保存的数据将被压缩。

按时间命名：勾选后，数据文件名将按系统时间命名；否则，文件名将按数字递增。

文件数量限制：设为 0 表示不限制文件数量，直至磁盘满；大于 0 时，将仅保存指定数量的文件。

文件大小限[2]制：设置传感器数据单个文件的最大大小（MB），0 表示不限制。

3.2.2 目标检测参数

距离门数量：用于设定目标检测扇形区的边界，需小于 1D FFT 输出大小。

检测范围：设置检测的目标范围，仅在此范围内的目标会被检测。

间距（毫米）：

[2] 部分操作系统不支持保存 2 G 以上大小的文件，限制文件大小为 2 G 以下可以避开此问题。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- 方位角：调整天线方位角间距。
- 俯仰角：调整天线俯仰角间距。

天线反相：

- 方位角：勾选后，方位角将反相。
- 俯仰角：勾选后，俯仰角将反相。

背景消除：

- 累计平均去背景：使用 ACC 算法去背景（仅 RAW、FFT 数据类型有效）。
- 去噪底：使用输入的系数进行去噪底操作。
- 去杂波：使用 cluster remove 算法去背景（仅 RAW、FFT 数据类型有效）。

目标能量阈值：不支持修改。

检测算法：不支持修改。

3.2.3 协议交互参数

固件协议类型：设置固件协议号，默认为 V2。

- **寄存器：**勾选后，打开设备时将设置芯片寄存器参数。
- **FPGA 寄存器：**不支持修改。
- **硬件信息：**勾选后，打开设备时将获取子板版本号信息。
- **驱动类型：**勾选后，打开设备时将设置驱动类型。
- **系统参数：**勾选后，打开设备时将设置系统参数（如 FFT 补零、级联模式等）。

保存：点击后生成应用配置文件。

3.3 常用参数说明

3.3.1 带宽配置功能

扫频带宽配置：当波形选择为“连续波+锯齿波”或“连续波+三角波”时，可以配置扫频频率范围，由起始频率和截止频率决定。例如，配置 1 GHz 带宽时，起始频率设为 24000 MHz，结束频率设为 25000 MHz。

连续波频率：当波形选择为“连续波”时，可以配置扫频频率，由起始频率决定。例如，配置 24G 频率时，起始频率设为 24000 MHz。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

3.3.2 链路增益配置

发射功率配置：选择开启或关闭指定发射通道。通道开启后，可配置通道的发射功率（或 EIRP）值。

接收增益配置：选择开启或关闭指定接收通道。通道开启后，可配置通道的接收增益值。

3.3.3 波形配置

波形模式：支持“连续波”、“锯齿波+连续波”、“三角波+连续波”三种模式。

Chirp 总时间：等于（T0+T1+T2+T3）。

T0 时间：Chirp 启动时间。

T1 时间：扫频频率上升时间（在“连续波+锯齿波”或“连续波+三角波”模式下）。

注意：每 Chirp 采样数必须大于 1D-FFT 计算量乘以取点间隔加偏移，否则将从 T2 周期获取数据补足。

T2 时间：扫频频率下降时间。

注意：在“连续波+锯齿波”模式下，T2 时间必须大于 T1 时间的四分之一。

T3 时间：Chirp 停止时间。

每帧 chirp 数：单个帧包含的 chirp 波形个数（此数量为所有发射的总 Chirp 数）。

T_PRE 时间：每帧调制启动时间。

T_NOP 时间：每帧调制结束等待时间。

T_2PD 时间：芯片进入低功耗模式时间，不小于 22 μs。

3.3.4 数据输出配置

数据输出格式支持 DSRAW、1DFFT、2DFFT、2DFFT_PEAK 等格式。表 3-1 详细说明了 RAW 数据取样、FFT 计算及输出等参数的设置要求。

表 3-1 数据处理参数说明

数据处理参数	参数说明
RAW 取点偏移	原始数据取样偏移
RAW 取点间隔	原始数据取样间隔
RAW 取点数量	原始数据取点数量
FFT 计算量	芯片 1DFFT 运算量
FFT 输出列偏移	1DFFT 数据偏移的距离单元数
FFT 输出列数量	1DFFT 结果偏移后输出的 FFT 数据量； 1DFFT 输出列数量 + 1DFFT 列偏移必须 ≤ 1DFFT 计算量
FFT 输出行偏移	设置输出的 1DFFT 的起始 chirp

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

FFT 输出行数量	1DFFT 运算完成后输出的 1DFFT 的 chirp 数； 1DFFT 输出行偏移 + 1DFFT 输出行数量 必须 ≤ 每帧 chirp 数
DPL 计算量（行数）	2DFFT 运算的 Chirp 数量
DPL 输出列偏移	2DFFT 运算完成后，设置输出的 2DFFT 的起始列
DPL 输出列数量	2DFFT 运算完成后，设置输出的 2DFFT 的列数； 2DFFT 输出列偏移 + 2DFFT 输出列数量 必须 ≤ 1DFFT 输出列数量
DPL 输出行偏移	2DFFT 运算完成后，设置输出的 2DFFT 的起始行
DPL 输出行数量	2DFFT 运算完成后，设置输出的 2DFFT 的行数； 2DFFT 输出行偏移 + 2DFFT 输出行数量 必须 ≤ 2DFFT 计算量（行数）

4 数据格式

4.1 芯片数据格式

芯片数据由 4 字节（32 位）数据头 + N 字节数据段 + 4 字节（32 位）数据尾组成。正常情况下，每个通道的数据通过一根 Tspi 数据线传输，格式如表 4-1 所示。

表 4-1 芯片数据格式

帧头	数据区				帧尾
tspi0_head0	tspi0_d0	tspi0_d1	…	tspi0_dn-1	tspi0_tail0
tspi1_head0	tspi1_d0	tspi1_d1	…	tspi1_dn-1	tspi1_tail0

同时，芯片支持将 2 路 Tspi 输出合并到一根 Tspi 数据线，合并方式如表 4-2 所示。

表 4-2 二合一数据格式

帧头	帧头	数据区							帧尾	帧尾	帧头	帧头
tspi0_head0	tspi1_head0	tspi0_d0	tspi1_d0	tspi0_d1	tspi1_d1	…	tspi0_dn-1	tspi1_dn-1	tspi0_tail0	tspi1_tail1	tspi0_head1	tspi1_head1

4.1.1 帧头（Head）格式

S5KM312L 芯片仅支持 1 种帧头（Head）格式，如表 4-3 所示。

表 4-3 S5KM312L 帧头格式

数据项	31:24	23:22	21:20	19:18	17:16	15:11	10:9	8:6	5:0
dfftpeak_head	aa	1	0	0	0	0			cfg_len_rpt[4:0]+1
fft0_head	aa	0	3	fft_chirp_cnt[8:0]			0		cfg_fft_tx_max[8:0]+1
fft1_head	aa	1	3	fft_chirp_cnt[8:0]			0		cfg_fft_tx_max[8:0]+1
adc0_head	aa	0	2	raw_chirp_cnt[8:0]			2(cfg_snum[2:0]+6)+1		
adc1_head	aa	1	2	raw_chirp_cnt[8:0]			2(cfg_snum[2:0]+6)+1		
dfft0_head	aa	2	3	3	3	dpl_frame_cnt[15:0]			
dfft1_head	aa	3	3	3	3	dpl_frame_cnt[15:0]			

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

说明:

bit[24~31] 数据头，固定为 0xAA

bit[22~23] 表示 RX 通道号，0-Rx0, 1-Rx1

bit[20~21] 表示数据类型， 2-dsraw, 3-fft

bit[11~19] 表示 chirp count

bit[0~8] 表示数据采样量+1，采样数据量为 bit[0~8]

ICL111A/ICL1112/ICL1122 芯片在上述格式基础上支持第二种 Head 格式（默认使用），如表 4-4 所示。

表 4-4 ICL111A/ICL1112/ICL1122 系列芯片默认帧头格式

数据项	31:24	23	22:20	19:12	11	10:0
dfftcfar_head	0xAA	1	0	0	0	dw_len+1
fft0_head	0xAA	0	3	fft_chirp_cnt[7:0]	fft_chirp_cnt[8]	dw_len+1
fft1_head	0xAA	1	3	fft_chirp_cnt[7:0]	fft_chirp_cnt[8]	dw_len+1
adc0_head	0xAA	0	2	raw_chirp_cnt[7:0]	raw_chirp_cnt[8]	dw_len+1
adc1_head	0xAA	1	2	raw_chirp_cnt[7:0]	raw_chirp_cnt[8]	dw_len+1
dfft0_head	0xAA	0	4	dpl_frame_cnt[7:0]	dpl_frame_cnt[8]	dw_len+1
dfft1_head	0xAA	1	4	dpl_frame_cnt[7:0]	dpl_frame_cnt[8]	dw_len+1

说明:

bit[24~31] 数据头，固定为 0xAA

bit[23] 表示 RX 通道号，0-Rx0, 1-Rx1

bit[20~22] 表示数据类型，0-dfft-cfar, 1-fft-peak, 2-dsraw, 3-fft, 4-dfft

bit[12~19], bit[11] 表示 chirp count，组合成 9bit 数据格式，

其中 bit[11]为 chirp_ count _bit[8], bit[12~19]为 chirp_ count _bit[0~7]。

当设置的 chirp index > 512 时，chirp index 会在 511 后变为 0 累加，

此时 Tailer 段的 frame cnt 保持不变

bit[0~10] 表示数据采样量+1，采样数据量为 bit[0~10]

ICL1240 芯片 Head 格式，如表 4-5 所示。

表 4-5 ICL1240 系列芯片帧头格式

数据项	31:24	23:22	21:12	11:0
dsraw_head	0xAA	0	dsraw_chirp_cnt -1	dw_len-1
iir_head	0xAA	1	iir_chirp_cnt -1	dw_len -1
fft_head	0xAA	2	fft_chirp_cnt -1	dw_len -1
dfft_head	0xAA	3	8'h0,tx_num[1:0]	dw_len -1

说明:

bit[24~31] 数据头，固定为 0xAA

bit[22~23] 表示数据类型，0:DSRAW, 1:IIR_Raw, 2:FFT, 3:DFFT

bit[12~21] 数据类型为 DSRAW/IIR_RAW/FFT 时： 表示 chirp count。当设置的 chirp index > 1024 时，chirp index 会在 1023 后变为 0 累加，此时 Tailer 段的 frame cnt 保持不变。

数据类型为 DFFT 时，表示当前数据的 tx 序号

bit[0~11] 表示数据采样量-1。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

4.1.2 帧尾（Tail）格式

与 Head 格式类似，S5KM312L 芯片仅支持一种帧尾（Tail）格式，如表 4-6 所示。

表 4-6 S5KM312L 帧尾格式

数据项	31:16	15:14	13:12	11:8	7:0
fftpeak_tail	check_sum	0	1	cfg_len_rpt[3:0]+1	55
dfftpeak_tail	check_sum	1	0	0	55
fft0_tail	check_sum	0	3	fft_chirp_cnt[3:0]	55
fft1_tail	check_sum	1	3	fft_chirp_cnt[3:0]	55
adc0_tail	check_sum	0	2	raw_chirp_cnt[3:0]	55
adc1_tail	check_sum	1	2	raw_chirp_cnt[3:0]	55
dfft0_tail	check_sum	5555			
dfft1_tail	check_sum	5555			

说明:

bit[16~31] 校验和，为数据段 16-bit 无符号数据累加

bit[14~15] 表示 RX 通道号，0-Rx0, 1-Rx1

bit[12~13] 表示数据类型， 2-dsraw, 3-fft

bit[8~11] 表示帧号

bit[0~7] 数据尾，固定尾 0x55

ICL111A/ICL1112/ICL1122 芯片在上述格式基础上支持第二种 Tail 格式（默认使用），如表 4-7 所示。

表 4-7 ICL111A/ICL1112/ICL1122 系列芯片默认帧尾格式

数据项	31:16	15:12	11:10	9:8	7:0
dfftcfar_tail	check_sum	dpl_frame_cnt[3:0]	RSV 0	cfg_msg[1:0]	0x55
fft0_tail	check_sum	frame_cnt[3:0]	Phase[1:0]	cfg_msg[1:0]	0x55
fft1_tail	check_sum	frame_cnt[3:0]	Phase[1:0]	cfg_msg[1:0]	0x55
adc0_tail	check_sum	frame_cnt[3:0]	RSV 0	cfg_msg[1:0]	0x55
adc1_tail	check_sum	frame_cnt[3:0]	RSV 0	cfg_msg[1:0]	0x55
dfft0_tail	check_sum	dpl_frame_cnt[3:0]	RSV 0	cfg_msg[1:0]	0x55
dfft1_tail	check_sum	dpl_frame_cnt[3:0]	RSV 0	cfg_msg[1:0]	0x55

说明:

bit[16~31] 校验和，为数据段按照 16-bit 无符号数据累加

bit[12~15] 表示帧号，循环累加。

bit[10~11] 保留

bit[8~9] 用户设置数据，可通过寄存器设置。上位机工具在芯片级联时，用于设置芯片号。

bit[0~7] 数据尾，固定尾 0x55

ICL1240 Tail 格式，如表 4-8 所示。

表 4-8 ICL1240 系列芯片帧尾格式

数据项	31:16	15:12	11:10	9:8	7:0
dsraw_tail	check_sum	frame_cnt -1	0	cfg_msg[1:0]	0x55

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

iir_tail	check_sum	frame_cnt -1	0	cfg_msg[1:0]	0x55
fft_tail	check_sum	frame_cnt -1	0	cfg_msg[1:0]	0x55
dfft_tail	check_sum	frame_cnt -1	0	cfg_msg[1:0]	0x55

说明:

bit[16~31] 校验和，为数据段按照 16-bit 无符号数据累加

bit[12~15] 表示帧号，循环累加。

bit[10~11] 保留

bit[8~9] 用户设置数据，可通过寄存器设置。上位机工具在芯片级联时，用于设置芯片号。

bit[0~7] 数据尾，固定尾 0x55

4.1.3 数据段格式

4.1.3.1 DSRAW 数据格式

S5KM312L/ICL111A/ICL1112/ICL1122:

双字	数据项	31:16	15:0
DW_0 : DW_{m-1}	Data	raw data real 0 [15:0]	raw data image 0 [15:0]
		raw data real 1 [15:0]	raw data image 1 [15:0]
		…	…
		raw data real m-1 [15:0]	raw data image m-1 [15:0]

ICL1240:

双字	数据项	31:16	15:0
DW_0 : DW_{m-1}	Data	raw data 0 [15:0]	raw data 1 [15:0]
		raw data 2 [15:0]	raw data 3 [15:0]
		…	…
		raw data 2m-2 [15:0]	raw data 2m-1 [15:0]

4.1.3.2 FFT 数据格式

双字	数据项	31:16	15:0
DW_0 : DW_{m-1}	Data	fft real data 0 [15:0]	fft image data 0 [15:0]
		fft real data 1 [15:0]	fft image data 1 [15:0]
		…	…
		fft real data m-1 [15:0]	fft image data m-1 [15:0]

4.1.3.3 2DFFT 数据格式

双字	数据项	31:16	15:0
DW_0 : DW_{m-1}	Data	dfft real data 0 [15:0]	dfft image data 0 [15:0]
		dfft real data 1 [15:0]	dfft image data 1 [15:0]
		…	…
		dfft real data m-1 [15:0]	dfft image data m-1 [15:0]

4.1.3.4 2DFFT_PEAK 数据格式

2DFFT_PEAK 默认将两个 Channel 的数据合并到一根 Tspi 线上传输，格式如下。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

双字	数据项	31	30:24	23	22:16	15	14:8	7	6:0
DW_1 : DW_{27}	Data	0	dfft0_max0_didx	0	dfft0_max0_ridx	0	dfft1_max0_didx	0	dfft1_max0_ridx
		dfft0_max0_value							
		dfft1_max0_value							
		0	dfft0_max1_didx	0	dfft0_max1_ridx	0	dfft1_max1_didx	0	dfft1_max1_ridx
		dfft0_max1_value							
		dfft1_max1_value							
		0	dfft0_max2_didx	0	dfft0_max2_ridx	0	dfft1_max2_didx	0	dfft1_max2_ridx
		dfft0_max2_value							
		dfft1_max2_value							
		0	dfft0_max3_didx	0	dfft0_max3_ridx	0	dfft1_max3_didx	0	dfft1_max3_ridx
		dfft0_max3_value							
		dfft1_max3_value							
		0	dfft0_max4_didx	0	dfft0_max4_ridx	0	dfft1_max4_didx	0	dfft1_max4_ridx
		dfft0_max4_value							
		dfft1_max4_value							
		0	dfft0_max5_didx	0	dfft0_max5_ridx	0	dfft1_max5_didx	0	dfft1_max5_ridx
		dfft0_max5_value							
		dfft1_max5_value							
		0	dfft0_max6_didx	0	dfft0_max6_ridx	0	dfft1_max6_didx	0	dfft1_max6_ridx
		dfft0_max6_value							
		dfft1_max6_value							
		0	dfft0_max7_didx	0	dfft0_max7_ridx	0	dfft1_max7_didx	0	dfft1_max7_ridx
		dfft0_max7_value							
		dfft1_max7_value							
		0	dfft0_max8_didx	0	dfft0_max8_ridx	0	dfft1_max8_didx	0	dfft1_max8_ridx
		dfft0_max8_value							
		dfft1_max8_value							

4.1.3.5 2DFFT_CFAR 数据格式

dfft cfar 数据区	31:23	22:16	15:7	6:0
1	0	cfar0_pmax3_didx	0	cfar0_pmax3_ridx
2	cfar0_pmax3_value			
3	0	cfar0_pmax2_didx	0	cfar0_pmax2_ridx
4	cfar0_pmax2_value			
5	0	cfar0_pmax1_didx	0	cfar0_pmax1_ridx
6	cfar0_pmax1_value			
7	0	cfar0_pmax0_didx	0	cfar0_pmax0_ridx
8	cfar0_pmax0_value			

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

4.2 MCU 数据封装

MCU 透传芯片数据时，除了上述的原始数据封包方式外，还支持将数据内容打包后再上传，打包格式如表 4-9 所示。

表 4-9 MCU 数据打包格式

帧头 (4 字节, "ICLH")	长度 (2 字节, LE)	类型 (1 字节)	扩展字段 (1 字节)	数据	帧尾 (4 字节, "ICLT")
0x49,0x43,0x4C,0x48	Data 段长度	-	-	参考表 4-10	0x49,0x43,0x4C,0x54

表 4-10 Data 描述

数据描述	类型	扩展字段	数据格式
芯片数据	0x00	通道号（SPI 号）	参考芯片数据格式
日志数据	0x10	日志级别	1: 调试日志 2: 普通日志 3. 警告日志 4. 错误日志
温度	0x80	芯片序号	4 字节值(Code 值)
功率	0x81	Tx 序号	4 字节值(Code 值)
校准回包	0x82	校准类型 0x01:回波能量校准	参考回波能量校准结果格式

5 通信协议

5.1 协议概述

本协议规定，上位机向下位机发送命令包后，下位机必须回复相应的响应包。上位机根据响应包的内容判断操作状态。若在规定时间内未收到响应包或响应包格式错误，则视为操作失败。

本协议定义了以下关键要素：帧结构、帧发送顺序及帧内数据格式。

5.2 帧结构

如图 5-1 帧结构示意图所示，每个命令构成一个帧，帧由以下四部分组成：

- **帧头**：4 个字节，固定值为 0xFAFBFCFD。
- **帧内数据长度**：2 个字节，表示帧内数据的长度（不包括帧头和帧尾）。
- **帧内数据**：具体内容参考后续帧内数据格式说明。
- **帧尾**：4 个字节，固定值为 0x01020304。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。



图 5-1 帧结构示意图

5.3 帧数据发送顺序

本协议采用低字节在前、高字节在后的发送顺序。例如，发送一个 6 字节的帧内数据时，发送顺序如图 5-2 所示。



图 5-2 帧数据顺序

5.4 帧内数据

每一条发送包都有一个相应的响应包，发送包和响应包的帧内数据内容略有不同：

- **发送包**：如图 5-3 所示，帧内数据包括命令类型和命令数据。特定命令（如开始、结束命令）可能不包含命令数据。
- **响应包**：如图 5-4 所示，帧内数据包括返回命令类型和返回结果，两者均不可缺。返回命令类型是在发送包命令类型的基础上，将高字节的 bit0 设置为 1。例如，发送命令类型为 0x00FF 时，相应的返回命令类型为 0x01FF。

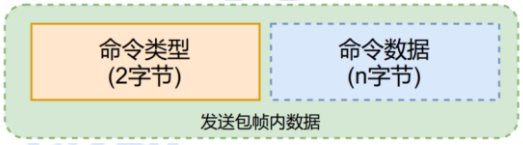


图 5-3 发送包帧内数据示意图

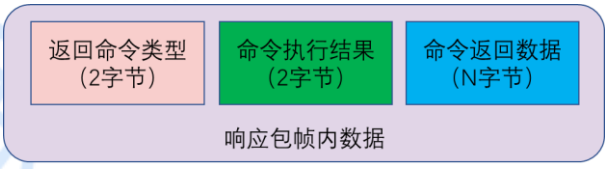


图 5-4 响应包帧内数据示意图

5.5 通信流程

毫米波传感器子板可通过串口或网口进行控制，控制方式包括命令控制与数据收发。毫米波传感器子板上电后默认处于 Idle 状态（部分固件可能默认处于数据传输或命令控制状态）。一般控制流程为：切换到命令模式；配置芯片设置；写入寄存器；切换到数据模式。下面是详细介绍。

1) 切换到命令模式

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

发送特定命令以切换设备状态。

示例发送数据[3]：FD FC FB FA 04 00 FF 00 01 00 04 03 02 01

成功 ACK：FD FC FB FA 08 00 FF 01 00 00 01 00 40 00 04 03 02 01

2) 配置芯片设置

根据应用需求发送配置命令。表 5-1 列举了部分常见命令。

表 5-1 常见配置命令与 ACK

命令	说明	发送命令数据	接收命令回包
设置降采样率	按 chirp 丢弃比例	FD FC FB FA 08 00 12 00 01 00 01 00 00 00 04 03 02 01	FD FC FB FA 04 00 12 01 00 00 04 03 02 01
设置寄存器格式	设置寄存器地址和数据位宽	16bit 地址 16bit 数据：FD FC FB FA 08 00 12 00 07 00 01 00 00 00 04 03 02 01 32bit 地址 32bit 数据：FD FC FB FA 08 00 12 00 07 00 02 00 00 00 04 03 02 01	FD FC FB FA 04 00 12 01 00 00 04 03 02 01
使能 MCU 数据打包模式	必须开启，否则多路数据会串包	FD FC FB FA 08 00 12 00 03 00 01 00 00 00 04 03 02 01	FD FC FB FA 04 00 12 01 00 00 04 03 02 01
设置 SPI 通道数量	支持 1/2 路	FD FC FB FA 08 00 12 00 0D 00 02 00 00 00 04 03 02 01	FD FC FB FA 04 00 12 01 00 00 04 03 02 01
启动前准备[4]	通知 MCU 即将启动芯片	FD FC FB FA 08 00 12 00 04 00 03 00 00 00 04 03 02 01	FD FC FB FA 04 00 12 01 00 00 04 03 02 01

3) 写入寄存器

按照三段式写入芯片寄存器。

发送数据格式：

FD FC FB FA 4*N+4 01 00 2-Byte 芯片地址（M-Byte 寄存器地址+M-Byte 寄存器值）* N 04 03 02 01

示例发送数据（16bit 地址 16bit 数据）：FD FC FB FA 18 00 01 00 20 00 72 00 D3 07 41 00 44 C8 00 00 00 20 01 00 64 8C 40 00 07 02 04 03 02 01

示例发送数据（32bit 地址 32bit 数据）：FD FC FB FA 3C 00 01 00 24 00 10 04 00 60 00 00 00 00 00 50 00 60 00 00 00 00 00 10 00 60 00 00 00 0C 00 70 00 60 42 06 00 07 04 70 00 60 04 00 00 00 00 F1 00 60 00 00 00 00 04 F1 00 60 00 00 00 00 04 03 02 01

成功 ACK：FD FC FB FA 04 00 01 01 00 00 04 03 02 01

N: 为单次发送的寄存器数量

M: 寄存器地址和数据位宽，16bit 为 2 字节，32bit 为 4 字节

[3] MSB（hex）。

[4] 上位机可以设置适当的延迟，以便等待 MCU 做准备工作。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

4) 切换到数据模式

发送特定命令以切换至数据模式。

示例发送数据：FD FC FB FA 02 00 FE 00 04 03 02 01

成功 ACK：FD FC FB FA 04 00 FE 01 00 00 04 03 02 01

5.6 通信示例

本节介绍基于串口毫米波传感器设备检测设备的通信流程。

5.6.1 读取设备版本

1) 进入命令模式

发送命令：FD FC FB FA 04 00 FF 00 02 00 04 03 02 01

首次响应（需丢弃，因为此时芯片处于数据发送状态）：回包数据（例如：00 01 00 02 00 15）

再次发送命令以确认进入命令模式：FD FC FB FA 04 00 FF 00 02 00 04 03 02 01

成功响应：FD FC FB FA 08 00 FF 01 00 00 03 00 00 02 04 03 02 01

解析结果：协议版本=3，发送/接收缓冲区大小=64

ICL_LOG 10:12:24.839 (pid:52804, lwp:56064) => protocol version = 3, send_recv buffer size = 64

2) 读取设备版本

发送命令：FD FC FB FA 02 00 00 00 04 03 02 01

响应：FD FC FB FA 0E 00 00 01 00 00 08 00 76 31 2E 35 2E 32 5F 44 04 03 02 01

解析结果：设备版本=v1.2.0_32768.0（注意：这里的版本格式可能因设备而异）

ICL_LOG 10:12:24.840 (pid:52804, lwp:56064) => ReadVersion Return: v1.2.0_32768.0

3) 退出命令模式

发送命令：FD FC FB FA 02 00 FE 00 04 03 02 01

响应：FD FC FB FA 04 00 FE 01 00 00 04 03 02 01

5.6.2 配置毫米波传感器

注意：读写寄存器的命令数据长度，不能超过 MCU send_recv buffer size。

1) 重新进入命令模式

同 5.6.1 小节的“1）进入命令模式”，发送命令并确认响应。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

2) **配置 MCU 参数**

- 设置 MCU 降采样间隔：

  发送命令：FD FC FB FA 08 00 12 00 01 00 01 00 00 00 04 03 02 01

  响应：FD FC FB FA 04 00 12 01 00 00 04 03 02 01

- 设置寄存器格式：

  发送命令：FD FC FB FA 08 00 12 00 07 00 02 00 00 00 04 03 02 01

  响应：同上

- 设置 MCU 数据封装：

  发送命令：FD FC FB FA 08 00 12 00 03 00 01 00 00 00 04 03 02 01

  响应：同上

- 设置 SPI 通道数量：

  发送命令：FD FC FB FA 08 00 12 00 0D 00 02 00 00 00 04 03 02 01

  响应：同上

- 写入停止、配置和开始寄存器：参考 5.5 节写入寄存器协议，具体寄存器值可由寄存器配置界面得到。

- 退出命令模式

  发送命令：FD FC FB FA 02 00 FE 00 04 03 02 01

  响应：FD FC FB FA 04 00 FE 01 00 00 04 03 02 01

  ICL_LOG 14:45:05.226 (pid:36584, lwp:23144) [I]=> Radar Control Start

6 毫米波传感器开发工具操作示例

本章将详细介绍如何使用毫米波传感器开发工具，以 DEV101 模组为载体，实现数据的透传功能。以下步骤将以输出 1DFFT 数据为具体示例，全面展示软件的操作流程。

1) **准备模组**：确保 DEV101 模组已完成固件烧录，并正确、稳定地连接到 PC 上。

2) **启动软件**：双击打开毫米波传感器开发工具软件，进入主界面。

3) **修改配置**：在主界面点击“修改配置”按钮，进入配置修改环节。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

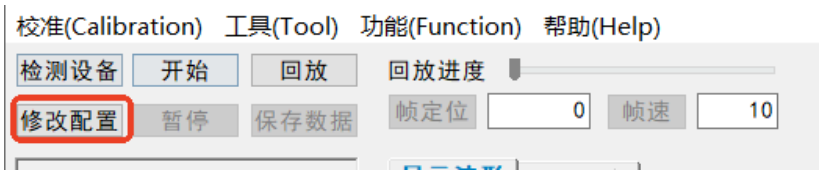


图 6-1 主界面“修改配置”按钮

4) **选择默认配置**：在弹出的“寄存器配置”窗口中，点击“选择默认配置”按钮，以快速加载预设配置。



图 6-2 “寄存器配置”界面的“选择默认配置”按钮

5) **生成默认配置**：在“加载默认配置”界面中，找到对应的 DEV101，点击“生成”按钮，加载该模组默认配置。



图 6-3 生成默认配置

6) **生成应用配置**：回到上一界面，点击“生成应用配置”按钮，进入“参数配置”界面。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。



图 6-4 生成应用配置

7) **保存配置**：在“参数配置”界面中，无需进行额外修改，直接点击“保存”按钮，将配置保存并生成。

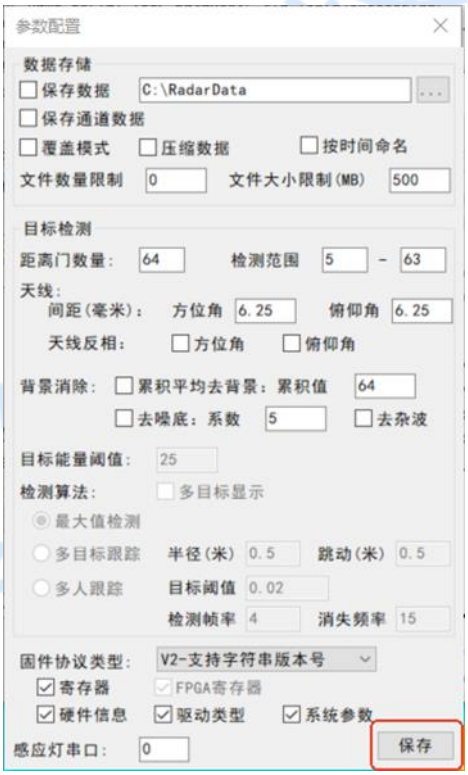


图 6-5 保存参数配置

8) **检测设备**：返回到主界面，点击“检测设备”按钮。若模组连接无误，系统将弹出提示框，显示“检测到串口

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

设备”。此时，点击“确认”按钮关闭提示框。

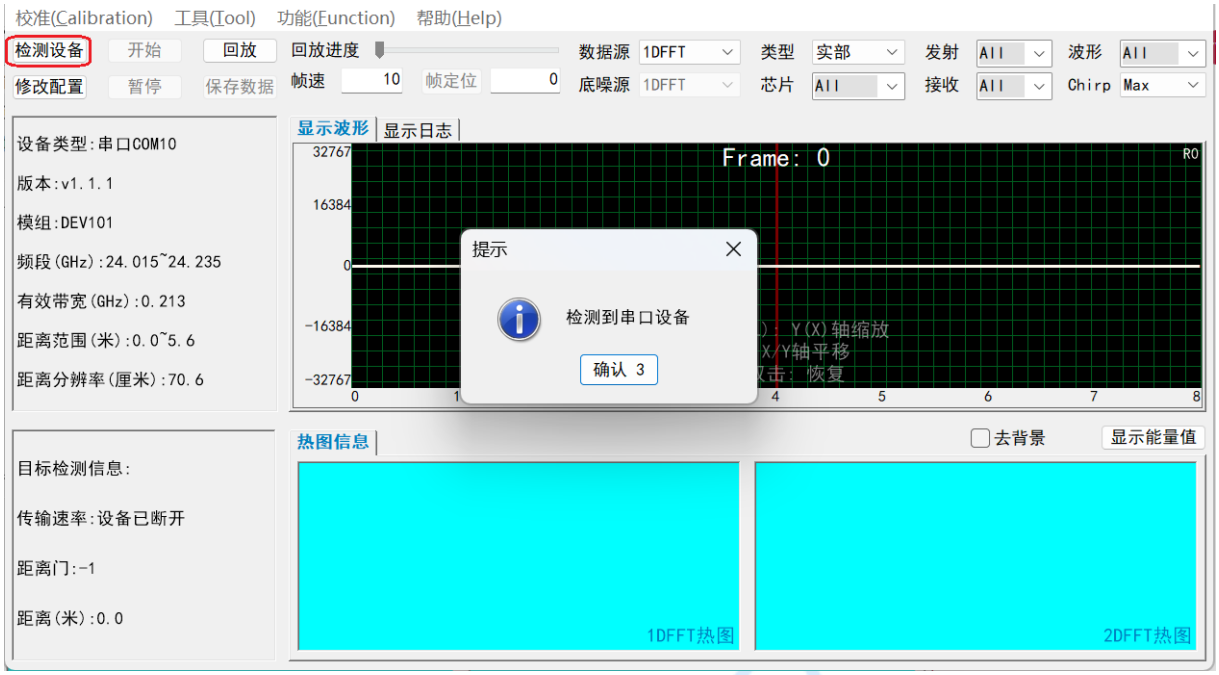


图 6-6 检测到设备

9) **启动与观察**：当主界面的“开始”按钮变为可点击状态时，点击它。稍等片刻，主界面将展示波形等关键信息，标志着数据透传操作已成功启动。

7 常见问题及解决方案

7.1 无法连接设备

当模组无法打开时的表现如下图 7-1：

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

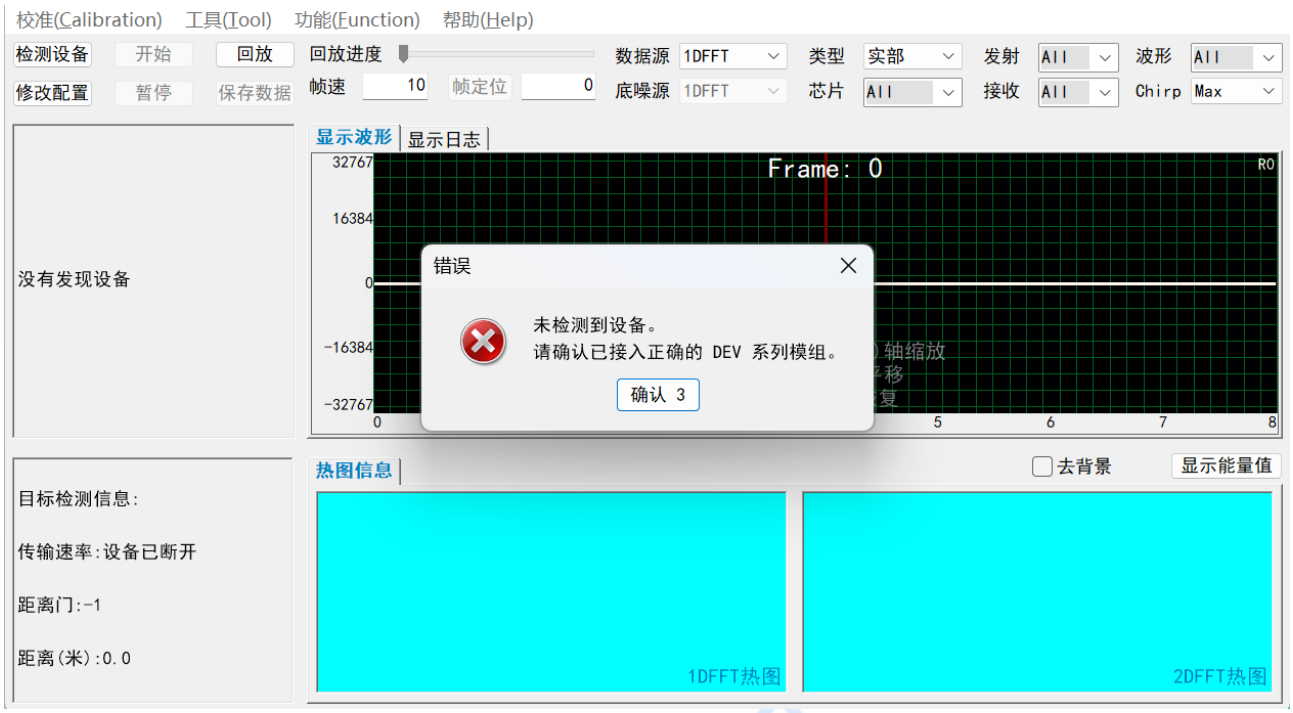


图 7-1 设备连接失败

一般可能有如下几种原因：

原因	解决方法
模组不是 DEV 系列	检查模组是否为 DEV 系列模组，避免与其他模组混淆
Pin 接错	参考模组用户手册，检查转接板与模组的接线是否正确
配置错误	选择对应模组默认配置
端口被占用	关闭其他可能打开该串口的软件
串口转接板功耗不支持	更换转接板或调整转接板输出功耗
固件非透传固件	重新烧录透传固件

7.2 配置后上位机不输出数据

在使用毫米波传感器开发工具修改芯片配置后，点击“开始”按钮，上位机界面中未查看到波形信息，或者在日志信息中观察到有丢包现象。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

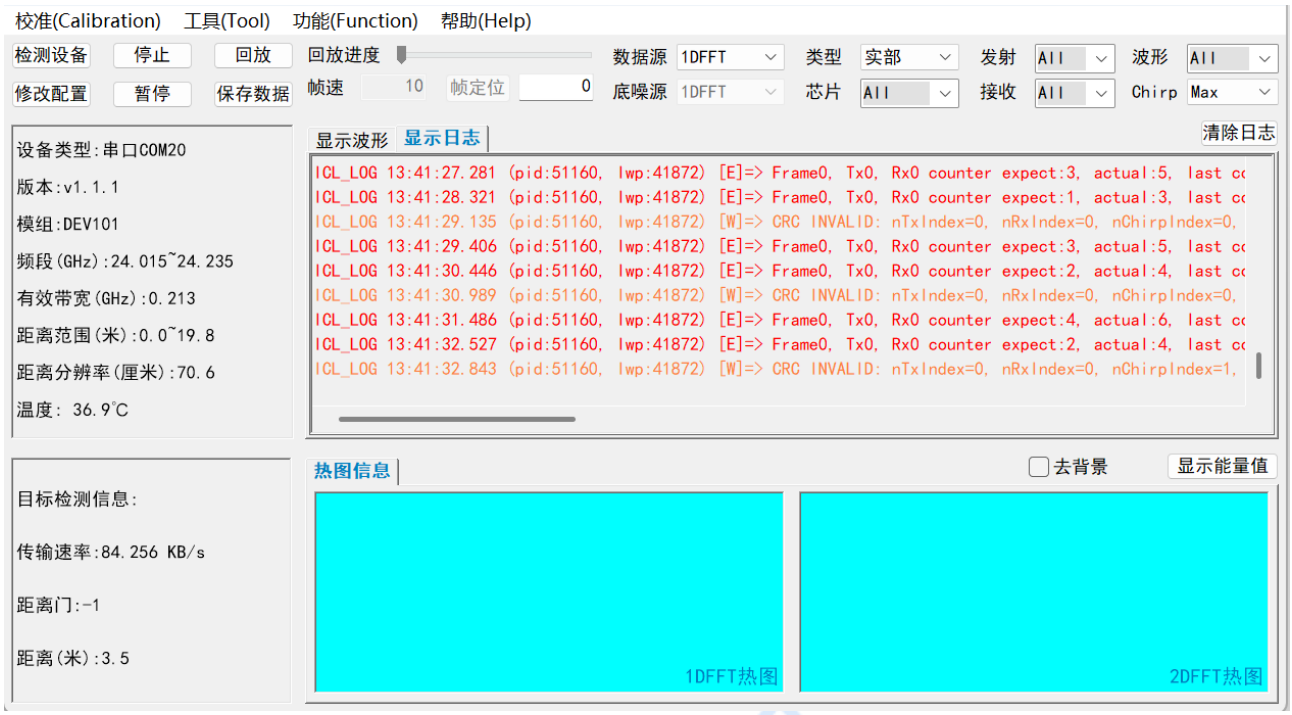


图 7-2 波形信息未显示界面示例

导致这一现象的原因有硬件连接、固件配置和传输速率匹配，下面详细介绍具体的问题与对应的解决方案。

7.2.1 设备连接问题

- **问题分析：**若设备配置接口与实际硬件接口不匹配，上位机将无法与设备进行正常通信，导致数据无法传输。
- **解决方案：**

  1. **核对数据源：**检查数据源是否与硬件输出方式一致；
  2. **检查串口信息：**确认软件中设置的串口号、波特率等参数与设备实际信息相匹配，如图 7-3 所示。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

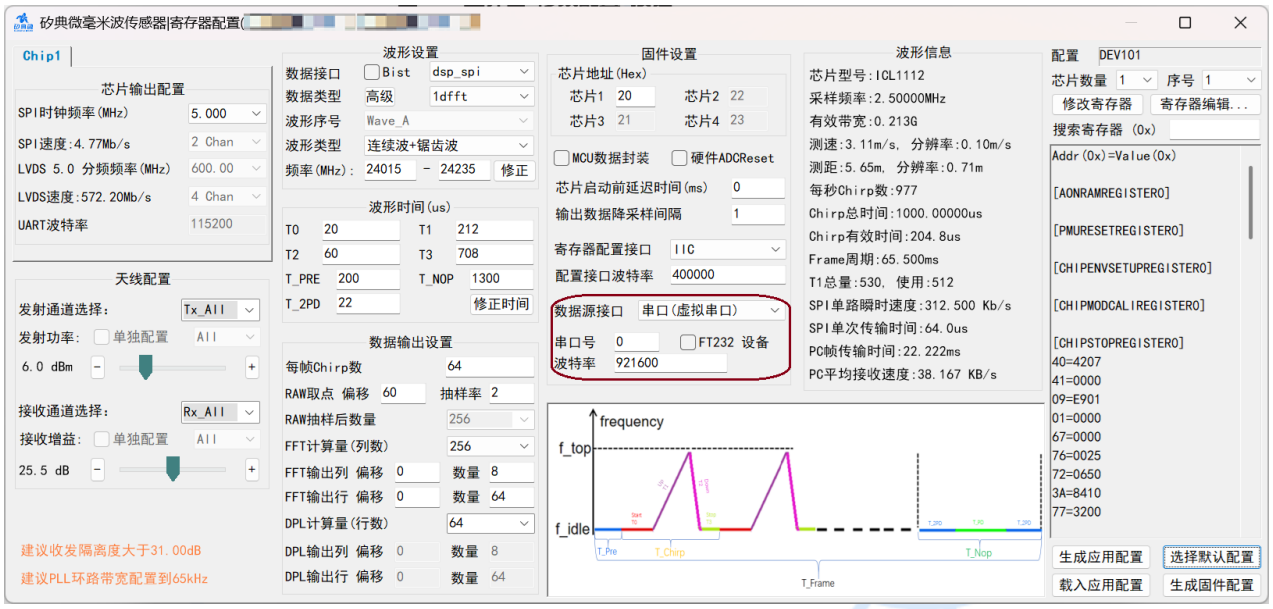


图 7-3 串口与设备信息

7.2.2 串口丢数据问题

- **问题分析**：部分使用串口输出的模组，在使用串口转接板连接电脑的时候，偶发丢数据现象。通过逻辑分析仪发现模组 Tx 数据正常，数据丢失发生在转接板或电脑驱动端。
- **解决方案**：

  1. **降低传输速度**：降低传输速度可有效避免数据丢失问题
  2. **提升电脑性能**：通过提升电脑性能可改善丢包现象

7.2.3 固件匹配问题

- **问题分析**：芯片配置信息超出其处理能力，或与外部 MCU 接口不兼容，导致外部 MCU 无法接收芯片数据。
- **解决方案**：

  1. **验证数据接口**：与 FAE（现场应用工程师）确认固件支持的数据接口，确保芯片数据接口与外部 MCU 相匹配。
  2. **确认数据类型**：考虑外部 MCU 的资源限制，与 FAE 确认固件支持的数据类型。
  3. **调整 SPI 时钟频率**：保芯片输出的 SPI 速率不超过外部 MCU 的处理能力，与 FAE 确认固件的 SPI 速率上限。如图 7-4 所示。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

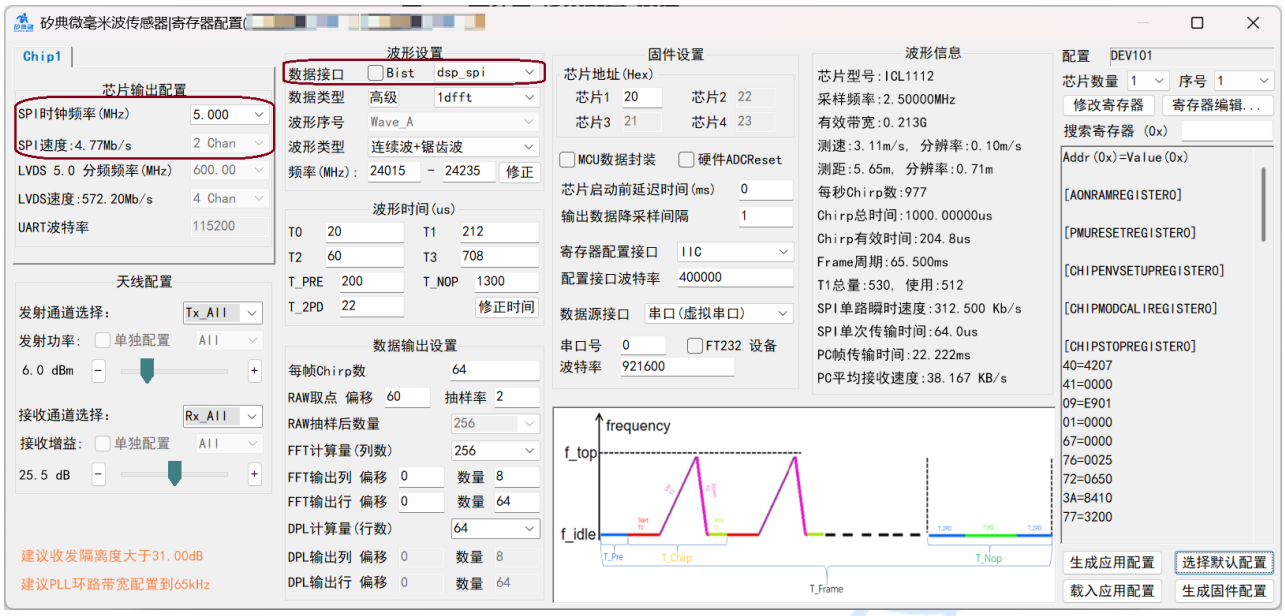


图 7-4 影响固件匹配的参数

7.2.4 芯片输出速率问题

- **问题分析**：芯片配置的传输速度超出其传输能力，导致数据输出不完整。
- **解决方案**：

  1. **增加 Chirp T3 时间**：降低芯片单个 Chirp 的传输速度（SPI 单路瞬时速度），确保其小于 SPI 速度。
  2. **减小数据输出量**：根据数据类型[5]（如 RAW、1DFFT、2DFFT），降低输出数据的抽样数量或点数，以减少数据负载。

7.2.5 固件输出速率问题

- **问题分析**：固件输出速度与硬件输出能力不匹配，导致数据丢包。
- **解决方案**：

  1. **核对接收速度**：确保 PC 的平均接收速度小于硬件输出的波特率[6]。
  2. **调整 T_Nop 时间**：对于部分 MCU 固件在帧间传输 Chirp 的情况，增加 T_Nop 时间，确保有足够的时间进行数据传输。
  3. **增加降采样间隔**：减少实际上报的 Chirp 数量，通过增加降采样间隔来降低数据输出频率。

[5] 部分芯片的 1DFFT/2DFFT 不支持修改 ROI（输出范围）。

[6] 对于串口设备，固件最大输出速度为 波特率 / 8 字节每秒；对于 USB 设备，如果使用的是 full speed，其传输速度一般 < 900 K 字节每秒；如果使用的是 high speed，其传输速度一般 < 22 M 字节每秒

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

7.3 如何计算实际输出的数据

输出速率是评估毫米波传感器系统性能的关键指标，它可以根据配置的波形参数以及数据输出格式进行计算。以下是详细的计算步骤。

7.3.1 Chirp 时间与 Frame 时间

- **Chirp 时间**：由 T0（静默时间）、T1（发射时间）、T2（接收延迟时间）和 T3（处理时间）组成，即 Chirp 时间 = T0 + T1 + T2 + T3。
- **Frame 时间**：一个完整的帧周期，包括帧前准备时间 T_Pre、Chirp 时间乘以 Chirp 数量以及帧后处理时间 T_Nop，即 Frame 时间 = T_Pre + Chirp 时间 * Chirp 数量 + T_Nop。

7.3.2 数据量计算

- **RAW 数据量**：每个 Chirp 的数据量由抽样后的 RAW 数据数量、数据位宽（通常为 16 bit）、复数表示（I + Q，即实部和虚部各占一半位宽）以及数据头尾（如校验码、时间戳等，这里假设为 8 字节）决定。

  计算公式为：（抽样后 RAW 数量 *2（16 bit） *2（I + Q） +8（数据头尾）） * 接收通道数。帧数据量则为 Chirp 数据量乘以 Chirp 数量。

- **1DFFT 数据量**：与 RAW 数据量类似，但需要考虑 1DFFT 的输出列数、数据位宽以及复数表示。

  计算公式为：（1DFFT 输出列 *2（16 bit） *2（实部 + 虚部） +8（数据头尾）） * 接收通道数。帧数据量则为 Chirp 数据量乘以 1DFFT 输出行数。

- **2DFFT 数据量**：2DFFT 通常按帧输出，每帧数据量由 2DFFT 输出列数、数据位宽、复数表示、接收通道数以及 2DFFT 输出行数决定。

  计算公式为：2DFFT 输出列 * 2（16 bit） * 2（实部 + 虚部） * 接收通道数 * 2DFFT 输出行 + 8（数据头尾）。

7.3.3 传输时间与速率计算

- **Chirp 传输时间**：数据量除以芯片输出速率（通常为 SPI 速率），即

  Chirp 传输时间 = 数据量 / 芯片输出速率

- **PC 传输时间**：帧数据量除以 PC 的传输速率（如 USB、以太网等），即

  PC 传输时间 = 帧数据量 / PC 传输速率

- **PC 平均传输速度**：帧数据量除以帧时间，即

  PC 平均传输速度 = 帧数据量 / 帧时间

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

7.3.4 计算举例

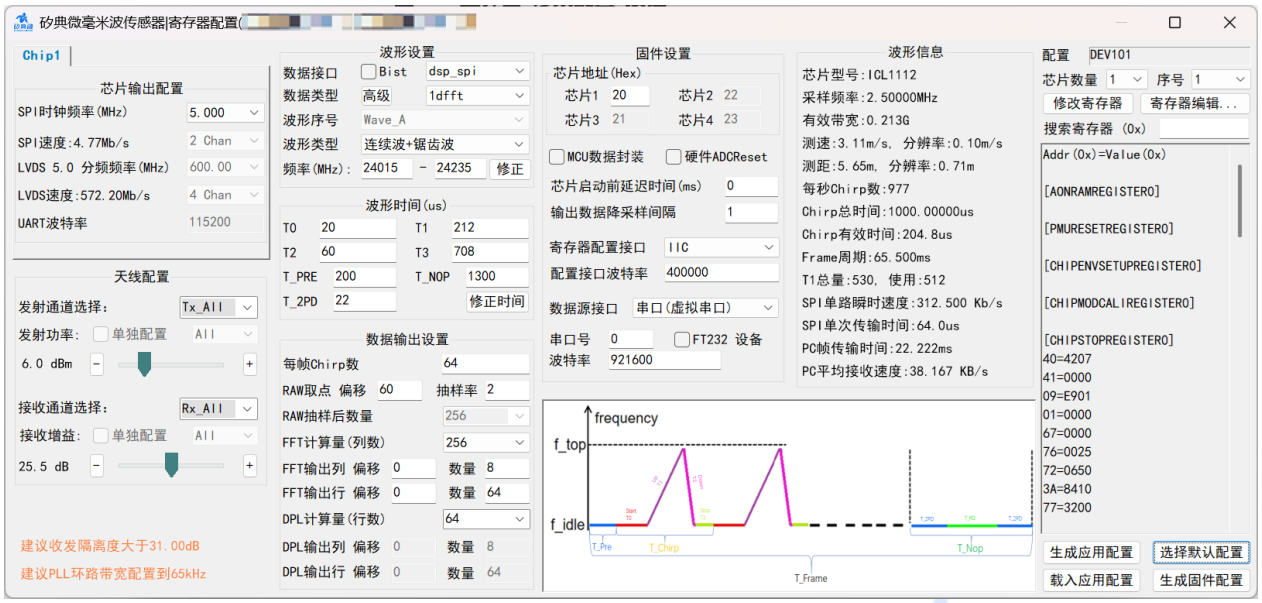


图 7-5 芯片配置示例

如图 7-5 所示，使用默认 DEV101 配置如下：

- 数据格式为 1DFFT。
- Chirp 时间为：T0 + T1 + T2 + T3 = 1000 μs。
- Frame 时间为：200 μs + 1000 μs * 64 + 1300 μs = 65500 μs = 65.5 ms。
- Chirp 数据量为：(8（输出列） *2（16 bit） *2（实部+虚部） +8（数据头尾）） *1（1 路接收通道）= 40 字节。
- 帧数据量为：64 * 40 = 2560 字节。
- 芯片 SPI 速率为 5 MHz（即 5000000 bps）。
- PC 传输速率为 921600 bps（即 921.6 kbps）。

则：

- Chirp 输出速率（瞬时输出速率）：40 字节 / 1000 μs = 40 * 8 / 0.001（bit/秒） = 320000.0（bit/秒）= 0. 305 Mbps。（320000.0/1024/1024~=0.305）
- Chirp 单次传输时间：40*8 bit / 5000000bps = 64.0 μs。
- PC 帧传输时间：2560*8 bit / 921600 bps = 0.022 s = 22.22 ms。
- PC 帧传输速度：2560 字节 / 65.5 ms = 39083.96 字节/秒 = 38.167 Kbps。（39083.96/1024~=38.167）

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

7.4 如何配置 Chirp 时间及采样信息

芯片在工作时，会根据配置的 Chirp 时间产生相应的波形，并在指定的采样范围和间隔中进行数据采样。为了确保数据采样达到预期目标，需要合理设置采样偏移、间隔和数量。

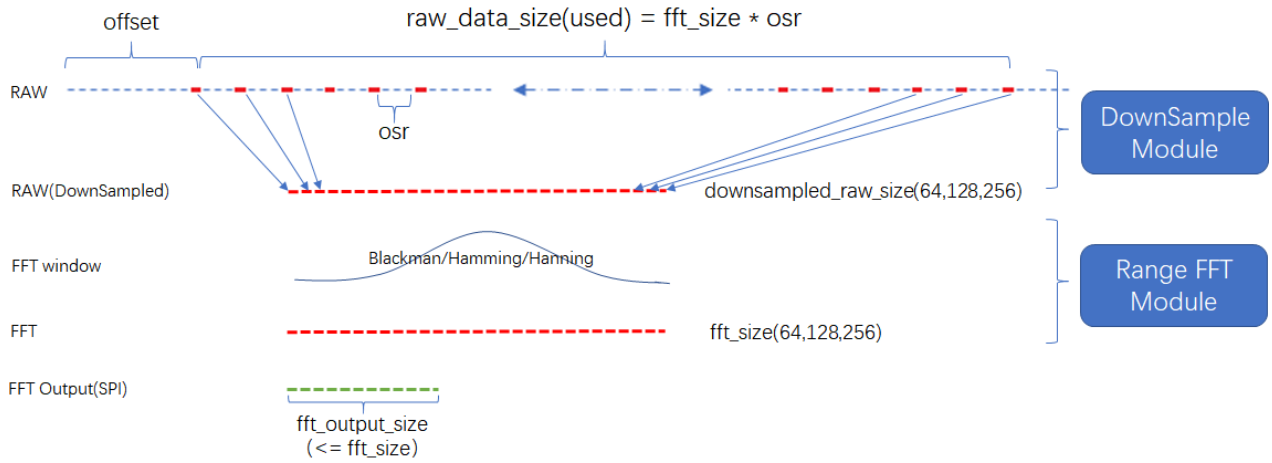


图 7-6 数据采样示例

7.4.1 配置步骤

1) **确定采样需求**：根据应用场景和算法要求，确定需要采集的 Chirp 时间段（如 T1）以及采样后的 FFT 点数和输出点数。

2) **计算采样参数**：

- 采样频率：由芯片规格决定，如 ICL111A 芯片的 ADC 采样频率为 2.5 MHz。
- 最小采样数量：根据 FFT 点数和采样频率计算得出，如 256 点 FFT 需要至少 256 个采样点。
- T1 时间：应大于最小采样时间，并考虑芯片低功耗模式下的额外时间需求。
- T2 时间：通常应大于 T1 时间的四分之一。
- T0 和 T3 时间：根据应用需求进行调整。

3) **设置采样偏移**：采样偏移量决定了数据采样的起始点。通常设置为 Chirp 时间段中点附近的某个值，以确保采集到完整且有效的信号。计算公式为：T0 时间 + (T1 时间 − 采样时间) / 2。注意将时间转换为对应的采样点数。

4) **配置芯片参数**：将计算得到的 Chirp 时间、采样偏移、采样间隔和数量等参数写入芯片寄存器，完成配置。

7.4.2 配置举例

7.4.2.1 1DFFT 输出配置

假设需求为：基于 ICL111A 芯片，采集 T1 时间的 RAW 数据，进行 256 点 FFT 后，输出前 64 个点的距离门

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

信息。

采样参数为：

- 采样频率：2.5 MHz。
- 采样抽样率：1（不抽样）。
- T1 时间：应大于 102.4 μs（256 点 / 2.5 MHz），建议设置为 110 μs。
- T2 时间：应大于 27.5 μs（110 μs / 4）。
- T0 和 T3 时间：假设 T0 = 20 μs，T3 = 100 μs。

采样偏移：23.8 μs（20 μs + (110 μs – 102.4 μs) / 2），转换为采样点数为 59.5，取整后为 60。

则芯片的参数配置如图 7-7 所示。



图 7-7 计算得到的芯片参数配置示例

7.4.2.2 DSRAW 输出配置

DSRAW 输出 RAW DATA，此时的数据量为配置中的 RAW 抽样后数量。Chirp 时间的计算与 1DFFT 计算方式相同。因为 RAW 数据量一般比较大，所以配置的时候特别需要主要保证如下两点：

- 芯片的输出速率不要超过 SPI 的时钟频率
- 模组输出给 PC 端的速率不要超过通信速率

以 DEV101 为例，如下图 7-8 配置可以稳定输出 DSRAW 数据：

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。



图 7-8 DSRAW 输出配置

8 自定义硬件开发

部分用户希望基于 ONELAB 开发套件搭建自己的开发平台（涵盖硬件与固件），可参考以下步骤进行部署。

8.1 硬件设计

用户根据所需的应用情景，在商城购买匹配该应用情景的 DEV 开发套件。购买后，将获得如下资料：

- 《xxx 硬件设计手册》：对应芯片的硬件设计手册。
- 《ONELAB 开发套件用户手册》：介绍 ONELAB 模组、工具使用以及固件开发说明。

8.2 MCU 选型

MCU 选型时，可参考如下指标进行评估：

1) **算力性能**: 推荐使用 cortex-m4 及以上级别的处理器。若任务所需资源（内存及算力）较低，也可考虑 Cortex-M0 级别处理器，需根据实际应用算法评估。

2) **RAM 大小**：MCU 需一定 RAM 以保证程序和算法执行。以透传固件为例，列出相关内存使用情况供参考（用户需根据自身应用场景，推导对应算法所需 RAM 大小）。

- SPI 数据接收模块：推荐使用 DMA 循环模式接收传感器芯片 SPI 数据，以提高数据处理效率。假设传感器芯片输出 N 点数据，则 SPI 数据接收模块所需内存大小为：2（DMA 缓存 2 倍大小）× （N × 4 + 8）× RX 数量，单位为 BYTE。例如，1T2R 模组（RX = 2）需输出 256 点 FFT，则数据接收 buffer 大小至少为：2 × （256 × 4 + 8）× 2 = 4128 BYTE。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- 数据解析和处理模块：此模块主要解析接收到的 SPI 数据。假设芯片输出 N 点数据，则数据解析模块所需内存大小为：（N × 4+8）× RX 数量，单位为 BYTE。（透传固件中解析完成后直接发送给上位机，无其他额外内存开销；若需在固件中进行其他数据处理逻辑，如各类算法，则需自行评估内存情况）
- Uart/Usb 交互模块（非必要模块）：参考模组一般包含命令交互模块和传感器数据上传模块。命令交互模块用于与上位机进行命令交互，需一个命令数据接收缓冲区和命令数据发送缓冲区，缓冲区内存大小理论上无限制，可根据开发情况自行定义。传感器数据上传模块用于将传感器数据上传至上位机，推荐将数据解析和处理模块解析后的数据拷贝到单独内存中上传，防止传感器数据覆盖，提高上传数据正确率。
- 其他模块：比如堆栈大小、引入的各种库函数、不同平台驱动或其他定制逻辑等，均为影响内存的重要因素。

3）FLASH 大小：MCU 一般自带内部 FALSH，用于存储和运行程序。FLASH 大小可综合考量以下因素：

- 程序文件大小：程序代码编译后的大小空间。
- 配置区大小：应用程序所需参数大小（比如保存的寄存器配置信息，厂测参数等）。
- 多分区切换：若应用程序需支持多个应用或 IAP（在线烧录）功能，需额外计算其他应用和 IAP 功能的程序大小。

4）处理器外设要求：

- 数据接收模块（可根据实际需求选择合适的数据接收方式）：
  - SPI（推荐）：推荐选择支持 25 Mbits/s spi 从模式的处理器，以匹配传感器芯片 SPI 最大输出速率；同时尽可能满足一个 RX 对应一路 SPI 的原则。例如，1T2R 模组需两路 SPI，2T4R 需 4 路 SPI。若 SPI 通道数不足，也可考虑采用 Merge 方式让芯片的两路 SPI 合并成一路 SPI 输出，此时芯片输出速度降为两路 SPI 输出的一半。有关 SPI MERGE 方式，可参考 3.1.1 芯片输出配置。
  - UART（可选）：传感器芯片支持使用 UART 115200 波特率输出数据。
- 配置接口（配置时序请参考传感器芯片 AN）：
  - IIC（推荐）：至少 1 路 IIC，默认 400K 时钟频率。
  - SPI(可选)：MCU 使用主模式配置。
  - UART(可选)：传感器芯片支持 115200 波特率的 UART 配置。
- 数据透传（可选：如需数据上传或协议交互）：
  - USB：如需大量数据的交互，可选择支持 USB 功能的 MCU 芯片。
  - UART：对于仅需协议交互或少量数据传输的情况，可选择支持所需波特率的 UART 功能的 MCU 芯片。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

8.3 固件移植

ONELAB 模组的开发套件中，包含透传固件源码和 ONELAB 开发套件用户手册，开发时可作参考。

8.3.1 开发环境搭建

1）MDK 工具安装：开发环境推荐使用 Keil 开发工具，推荐用户直接从官网下载软件进行安装，地址：https://www.keil.com/download/product/



图 8-1 MDK 工具下载界面

2）MCU 开发包安装：安装完工具后，需要安装对应 MCU 的开发包。MCU 开发包有两种安装方式：一是直接从官网进行下载；二是在 MDK 软件进行在线下载。以下以 PY32F4 系列为例，介绍两种安装方式的步骤。

方案一：官网下载:

下载地址：https://www.puyasemi.com/en/gongjuyuruanjian.html?keywords=403

如图 8-2 所示，点击“PY32F403_Keil_DFP”进行下载。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

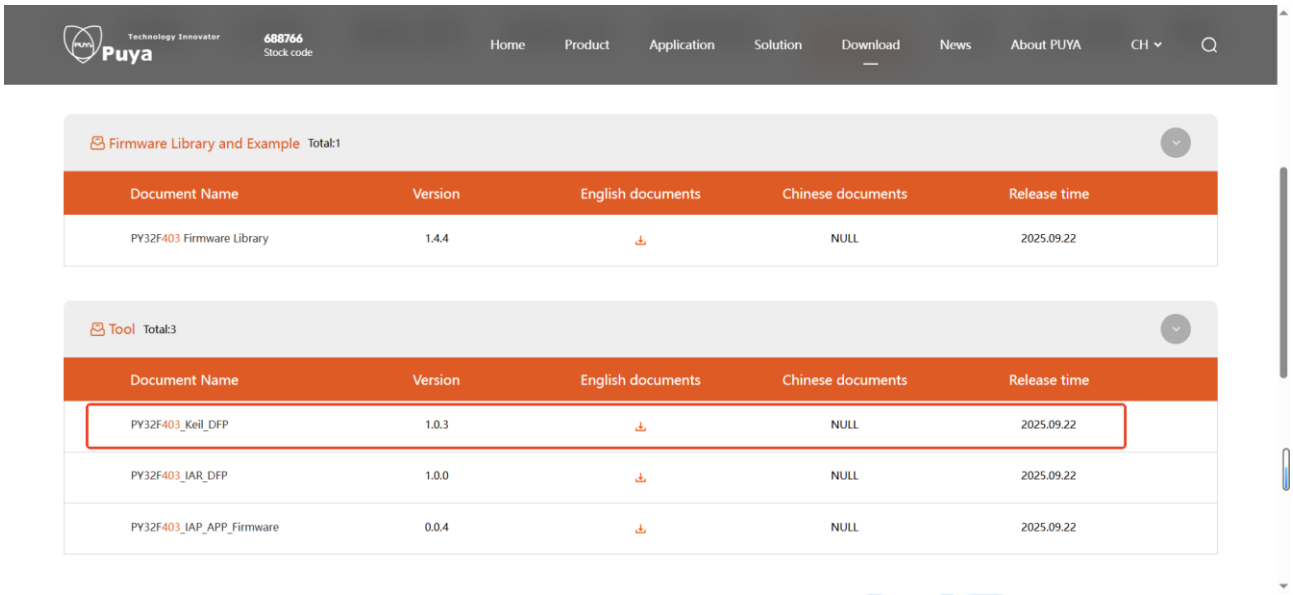


图 8-2 MCU 开发包安装

下载成功后，点击“Pack Installer”图标。



图 8-3 下载后点击安装图标

在 Pack Installer 安装界面，依次点击“File”，“Import”。

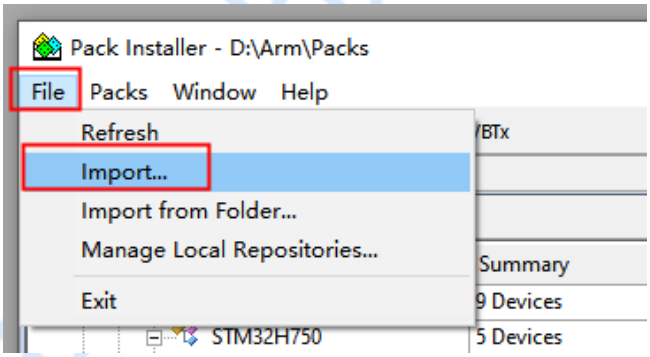


图 8-4 Pack Installer 安装界面

选择下载的.pack 文件，打开即可进行安装。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

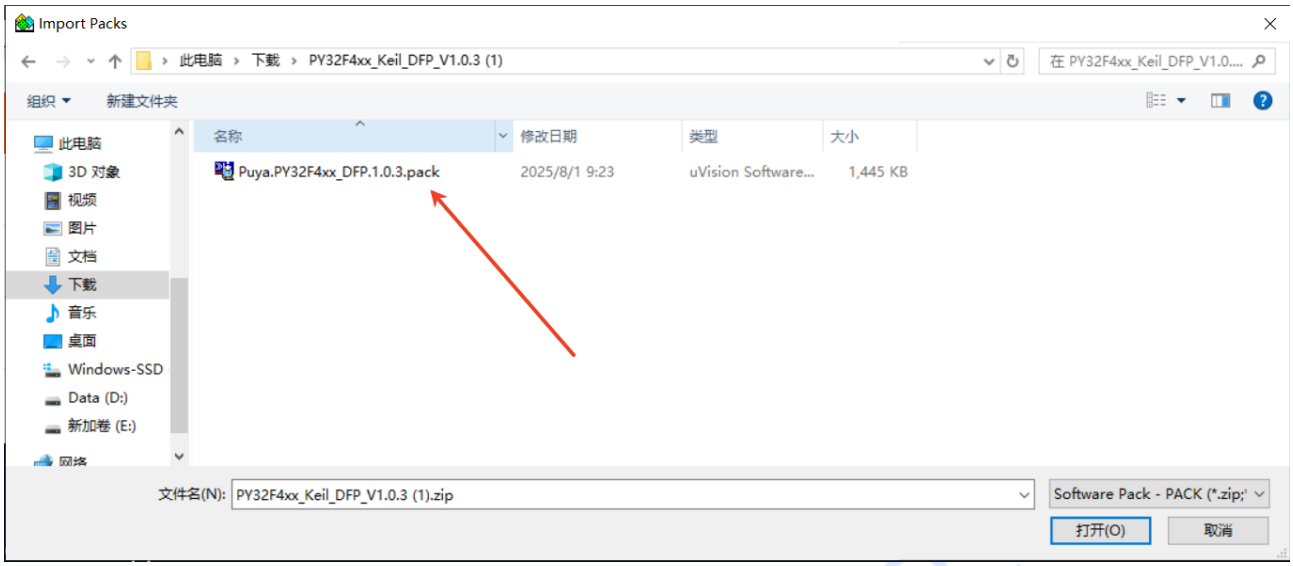


图 8-5 下载至本地

用户可在页面底部看到安装进度条。



图 8-6 安装界面底部的进度条

方案二：在线安装

点击“Pack Installer”图标，如图 8-3 所示。

在图示的 Devices 中，找到“Puya”，双击“PY32F4 Series”一栏，相应在右侧 “Packs” 中出现 “Puya::PY32F4xx_DFP” 一栏，点击 Installer 即可进行在线安装。用户可通过观察页面底部进度条查看安装进度。

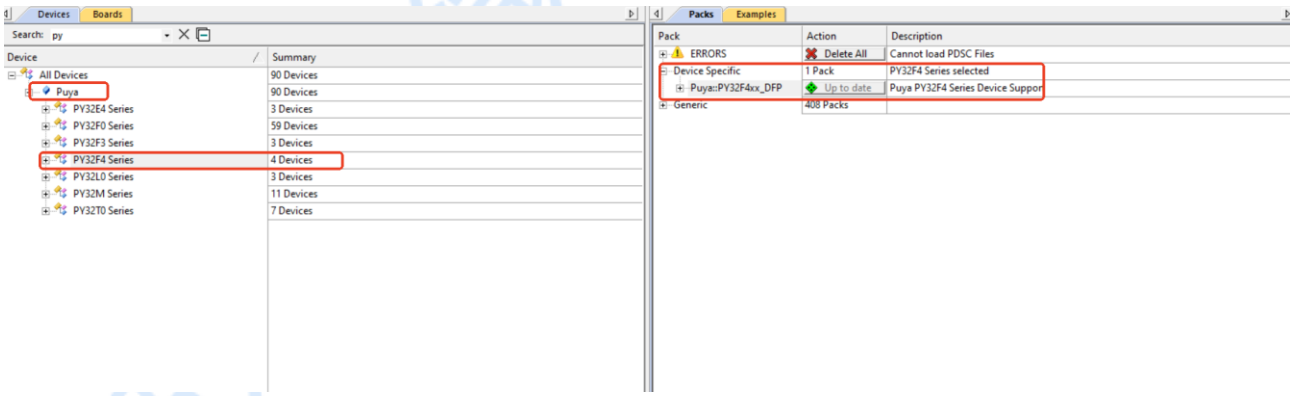


图 8-7 Devices 界面

至此，MDK5 开发环境已准备完毕。

8.3.2 代码开发调试

1）下载参考代码

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

ONELAB 模组的开发套件中包含透传固件源码，可直接用 Keil 编译运行。

若为自设计模组且更换了处理器平台，则可去相应处理器官网下载官方例程协助开发。

2）编译和下载代码

点击工具栏上的 Build 按钮（通常为锤子图标）编译项目。

若编译成功，点击 Program 按钮（通常为下载箭头图标）将代码下载到微控制器。

3）调试（可选）

若需调试代码，可设置断点，然后使用 Keil 的调试工具进行调试。点击工具栏上的 Debug 按钮（虫子图标）开始调试会话。

以上步骤应能帮助用户在 Keil MDK-ARM 中生成并运行一个基本的默认代码项目。若有特定模板或框架需求，可能还需进一步自定义或查找适合的库和示例代码。

8.3.3 平台相关代码的移植：

此部分包含不同处理器平台外设相关的代码移植，包括 SPI 模块、IIC 配置模块、UART 模块、中断模块、GPIO 模块等。其中 SPI 和 IIC 是常见接口方式。移植过程中需根据官方驱动、例程实现 SPI 从模式的传感器数据接收和 IIC 配置传感器寄存器的功能。

平台相关代码位于透传固件目录下的 Driver 文件夹和 platform 文件夹：

- Driver：存放不同处理器平台第三方官方的驱动文件；
- platform：基于第三方官方的 BSP 封装的适配不同平台的驱动代码。

注意：此部分代码更多需要参考官方提供的示例代码。若 MCU 型号与 ONELAB 模组使用的 MCU 型号相同，可参考 ONELAB 提供的固件代码。

8.3.4 平台无关代码的移植：

此部分代码一般可跨平台直接使用。主要包含上电配置传感器默认寄存器、上位机命令的收发和解析、传感器数据的解析和上传。移植过程中可根据需求增删功能。平台无关代码位于 App 文件夹和 Component 文件夹。

1）上电配置传感器默认寄存器：

开发板在上电后会给配置传感器默认寄存器，此配置定义在 banyan_para.c（Middleware\common\src）中。

以下介绍传感器芯片寄存器的默认配置及调整方法（以双芯片级联为例，若为单芯片则只需传感器芯片 0 的配置）：

- InitChipRegListConfig0 ： 传感器芯片 0 的 config 段配置
- InitChipRegListStart0 ： 传感器芯片 0 的 start 段配置
- InitChipRegListConfig1 ： 传感器芯片 1 的 config 段配置

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

- InitChipRegListStart1 ：传感器芯片 1 的 start 段配置

```
  2  static RADAR_REG_T InitChipRegListConfig0[MAX_REG_NUM] __attribute__((aligned (4))) =
  3 {
 72  static RADAR_REG_T InitChipRegListStart0[MAX_REG_FIX_NUM] __attribute__((aligned (4))) =
 73 {
 81  static RADAR_REG_T InitChipRegListConfig1[MAX_REG_NUM] __attribute__((aligned (4))) =
 82 {
151  static RADAR_REG_T InitChipRegListStart1[MAX_REG_FIX_NUM] __attribute__((aligned (4))) =
152 {
```

图 8-8 默认参数设置

若想调整 banyan_para.c 传感器芯片寄存器的默认配置，可修改 Config\radarPara ...\ regs.txt 文件，修改后需重新编译工程。格式说明如下：

```
ffff=0 //芯片0的config段，对应banyan_para.c文件中InitChipRegListConfig0
xxxxx
fff0=0

fffe=0 //芯片0的start段，对应banyan_para.c文件中InitChipRegListStart0
xxxxx
fff0=0

ffff=1 //芯片1的config段，对应banyan_para.c文件中InitChipRegListConfig1
xxxxx
fff0=1

fffe=1 //芯片1的start段，对应banyan_para.c文件中InitChipRegListStart1
xxxxx
fff0=1
```

图 8-9 传感器芯片寄存器的默认配置修改界面

配置传感器芯片采用三段式配置流程，即 Stop 段 => Config 段 => Start 段。

系统启动时，通过函数 Radar_Init_Stop_BanyanE()，将 Stop 段（数组 InitChipRegListStop_BanyanE）和 Config 段寄存器配置通过 RadarConifg_Write 接口逐个写入传感器芯片中，再通过函数 Radar_Init_Start_BanyanE()将 Start 段寄存器配置通过 RadarConifg_Write 接口逐个写入传感器芯片中。注意：级联情况下，每段配置过程中均先配置从芯片，再配置主芯片。

```
void Radar_Init_Stop_BanyanE(void)
{
    uint16_t loop = 0;
#ifdef TWO_CHIP
    while(InitChipRegListStop_BanyanE[loop].addr)
    {
        RadarConifg_Write(IIC_BANYAN_ADDR1, (uint8_t)(InitChipRegListStop_BanyanE[loop].addr),
InitChipRegListStop_BanyanE[loop].val);
        loop++;
    }
    loop = 0;
#endif
    while(InitChipRegListStop_BanyanE[loop].addr)
```


版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

```
    {
        RadarConifg_Write(IIC_BANYAN_ADDR, (uint8_t)(InitChipRegListStop_BanyanE[loop].addr),
InitChipRegListStop_BanyanE[loop].val);
        loop++;
    }

    loop = 0;
#ifdef TWO_CHIP
    while(InitChipRegListConfig1[loop].addr)
    {
        RadarConifg_Write(IIC_BANYAN_ADDR1, (uint8_t)(InitChipRegListConfig1[loop].addr),
InitChipRegListConfig1[loop].val);
        loop++;
    }
    loop = 0;
#endif
    while(InitChipRegListConfig0[loop].addr)
    {
        RadarConifg_Write(IIC_BANYAN_ADDR, (uint8_t)(InitChipRegListConfig0[loop].addr),
InitChipRegListConfig0[loop].val);
        loop++;
    }
}

void Radar_Init_Start_BanyanE(void)
{
    uint16_t loop = 0;

#ifdef TWO_CHIP
    while(InitChipRegListStart1[loop].addr || InitChipRegListStart1[loop].val)
    {
        RadarConifg_Write(IIC_BANYAN_ADDR1, (uint8_t)(InitChipRegListStart1[loop].addr),
InitChipRegListStart1[loop].val);
        loop++;
    }
    loop = 0;
#endif
    while(InitChipRegListStart0[loop].addr || InitChipRegListStart0[loop].val)
    {
        RadarConifg_Write(IIC_BANYAN_ADDR, (uint8_t)(InitChipRegListStart0[loop].addr),
InitChipRegListStart0[loop].val);
        loop++;
    }
}
```


版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

2）传感器数据的接收与处理

数据处理功能包含两个主要模块：数据接收模块和数据解析处理模块。数据解析处理模块主要的代码在dataprocess.c(App\common\src)，其中 DataProc_Recv 主要处理 spi 接收到的传感器数据，并将处理后的数据通过 UART 传输给上位机。

- **数据的接收（以 PY32F403 系列 MCU + ICL1122 芯片为例）**

传感器数据由 Spi 接口进行数据的接收，传感器芯片作为 spi master，mcu 作为 spi slave。为了提升代码运行效率和数据的正确性，采用了 DMA 双缓冲 buffer 机制搬运 Spi 数据。

ICL1122 芯片为 1T2R，会产生两路 SPI 数据。通常单芯片模式下需 MCU 提供两路 SPI 接收数据；双芯片级联模式下则需 4 路 SPI 接收数据。

两路 normal 模式传输流程如图 8-10 所示：

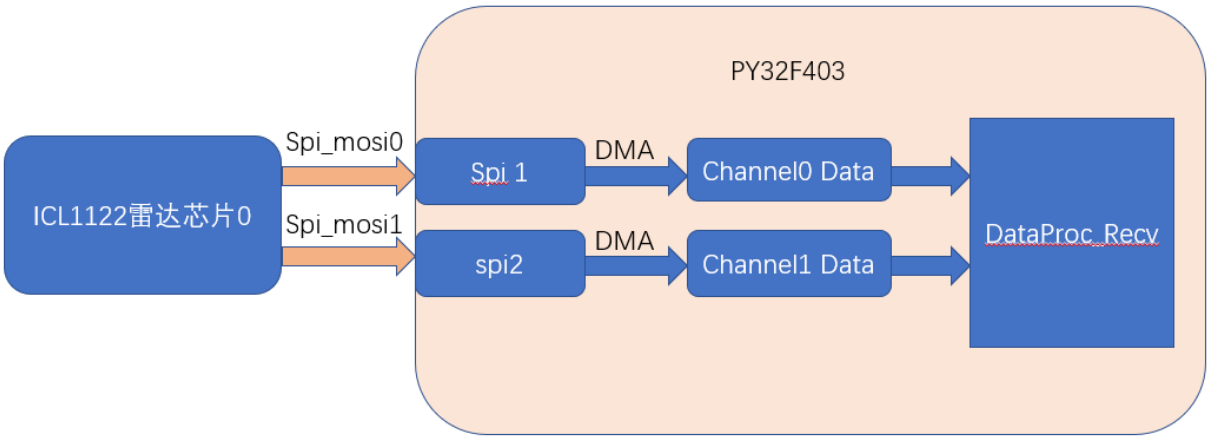


图 8-10 两路 normal 模式传输流程

每次调整传感器芯片配置（包含上电配置和上位机协议配置）后，传感器芯片对外输出的一个 chirp 或一个 frame 的数据量可确定，从而可准确设置 DMA 每次搬运的数据长度。

以上电配置为例进行说明，调用 DataProc_Init 函数。

```
void DataProc_Init(void)
{
.....
    GetRadarPara(&RadarPara);

.....

    SPI_Init(RadarPara.dataLen *2);
}
```

首先通过 GetRadarPara 获取数据长度，记录在 RadarPara.dataLen 变量中。并将两倍此长度用于 SPI DMA 的

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

初始化（双缓冲模式）。

SPI 初始化正确设置后，传感器芯片的数据通过 MCU SPI 接收，再由 DMA 搬运到指定的 RAM 缓存区。以 DEV202 开发板为例，接收数据使用 PY32F403 的 SPI1 和 SPI3，因此可在 HAL_SPI_RxHalfCpltCallback（半传输完成）和 HAL_SPI_RxCpltCallback（传输完成）中获取对应 SPI 的传感器数据。注意，一般中断处理程序中不做耗时操作，因此在回调函数中仅将对应通道的 Flag 置位。

```
void HAL_SPI_RxHalfCpltCallback(SPI_HandleTypeDef *hspi)
{
	if(hspi->Instance == SPIx)
	{
		if(DMA_FLAG_HT2 == READ_REG(DMA_ISR_HTIF2))
		{
			dataRecvFlag[CHANNEL_0][DMA_RECV_FLAG_MEM_0] = 1;
			SET_BIT(DMA1->IFCR,DMA_IFCR_CHTIF2);
		}
	}

	if(hspi->Instance == SPI3x)
	{
		if(DMA_FLAG_HT1 == READ_REG(DMA_ISR_HTIF1))
		{
			dataRecvFlag[CHANNEL_1][DMA_RECV_FLAG_MEM_0] = 1;
			SET_BIT(DMA2->IFCR,DMA_IFCR_CHTIF1);
		}
	}
}


void HAL_SPI_RxCpltCallback(SPI_HandleTypeDef *hspi)
{
	if(hspi->Instance == SPIx)
	{
		if(DMA_FLAG_TC2 == READ_REG(DMA_ISR_TCIF2))
		{
			dataRecvFlag[CHANNEL_0][DMA_RECV_FLAG_MEM_1] = 1;
			SET_BIT(DMA1->IFCR,DMA_IFCR_CTCIF2);
		}
	}

	if(hspi->Instance == SPI3x)
	{
		if(DMA_FLAG_TC1 == READ_REG(DMA_ISR_TCIF1))
		{
```


版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

```
                        dataRecvFlag[CHANNEL_1][DMA_RECV_FLAG_MEM_1] = 1;
                        SET_BIT(DMA2->IFCR,DMA_IFCR_CTCIF1);
                }
        }
}
```

工程中采用 DMA 的双缓存传输机制，即定义两组缓存用于接收 DMA 数据，dataRecvFlag 用于指示哪一组缓存收到数据。数据接收函数 DataProc_Recv 通过判断此标志获取对应通道数据。

```
void DataProc_Recv(void)
{
.....
    for (channel = 0; channel < CHANNEL_MAX; channel++)
    {
        for (dmaFlag = 0; dmaFlag < DMA_RECV_FLAG_MAX; dmaFlag++)
        {
            if (!Get_SPI_DataRecvFlag(channel, dmaFlag))
            {
                continue;
            }

.....

            Clr_SPI_DataRecvFlag(channel, dmaFlag);

            if (DMA_RECV_FLAG_MEM_0 == dmaFlag)
            {
                dataPos = 0;
            }
            else
            {
                dataPos = RadarPara.dataLen;
            }
            DataProcess(channel, dmaFlag, &g_dataRecvBuf[channel][dataPos], RadarPara.dataLen);
        }
    }
}
```

- **数据的处理**

上一小节中数据接收函数获取数据后，通过调用 DataProcess (data_process.c)对传感器数据进行处理。

在 DataProcess 函数中，调用 DataParse 解析收到的传感器数据，判断数据是否准确无误。数据正确前提下，再调用 DataDispatch 函数。此函数提供算法处理接口 AlgorithmProcessFun 和数据透传接口 StartDataTransfer，

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

可根据使用场景自行选择。

```
static void DataDispatch(uint8_t* frameBuf, uint16_t bufLen, uint8_t channel, uint16_t index)
{
    if (NULL == frameBuf || 0 == bufLen)
    {
        return;
    }

#if 0
    AlgorithmProcessFun(frameBuf, bufLen, channel, index);
#else
    StartDataTransfer(frameBuf, bufLen, channel, index);
#endif
}
```

在透传固件中，调用 StartDataTransfer 函数将降采样后的数据放入传输缓冲区，由 DataTransferPro 函数从传输缓冲区中取出数据调用 uart_transmit 将数据发送给上位机。

```
static void StartDataTransfer(uint8_t* frameBuf, uint16_t bufLen, uint8_t channel, uint16_t index)
{
....

    if(canSend)
    {
        if(System_UseMcuPackWrapper())
        {
            memcpy(TransferDataBuf[bufIndex], frameBuf, bufLen);
            uDataTransferProLen = bufLen;
        }
        else
        {
            memcpy(TransferDataBuf[bufIndex], frameBuf + 8, bufLen - 12);
            uDataTransferProLen = bufLen -12;
        }
        sendDataFlag[bufIndex] = 1;
        bufIndex = (bufIndex + 1) % 4;
    }
}

void DataTransferPro(void)
{
    if(!uart_is_Ready())
    {
```


版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

```
            return;
        }

        for(uint8_t loop = 0; loop < 4; loop++)
        {
            if(sendDataFlag[loop] == 1)
            {
                uart_transmit(TransferDataBuf[loop], uDataTransferProLen);
                sendDataFlag[loop] = 0;
                break;
            }
        }
}
```

3）上位机命令的收发和解析：

命令接收即下位机 MCU 接收来自上位机的命令数据。当下位机接收到正确命令数据并完成处理后，会回发给上位机相应响应数据包。协议格式可参考通信协议章节。

- **命令的接收**

DEV 系列模组均以 UART 方式接收上位机命令数据，固件中采用 uart 中断方式将接收数据保存在 g_uartCmdRecv.buf 中。当接收到的 UART 数据满足一包大小或超时 10ms 未再次收到 uart 数据时，将 g_uartCmdRecv.cmdReady 置 1，表示命令接收完成。

```
void UART_RecvTimeout(void)
{
    if (g_uartCmdRecv.curIndex > 0)
    {
        g_uartCmdRecv.idleCnt++;
        if (g_uartCmdRecv.idleCnt >= CMD_RECV_TIMEOUT)
        {
            g_uartCmdRecv.cmdReady = 1;
....
        }
    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    g_uartCmdRecv.buf[g_uartCmdRecv.bufRecv][g_uartCmdRecv.curIndex++] = aRxByte;
    g_uartCmdRecv.idleCnt = 0;
    if (g_uartCmdRecv.curIndex >= CMD_RECV_BUF_SIZE)
    {
        g_uartCmdRecv.cmdReady = 1;
```


版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

```
....
    }
}
```

- **命令的处理**

该部分代码在 cmd_process.c/h（分别位于 App\common 目录下的 src 和 inc 文件夹中）。

主函数中持续调用 CmdProc_Recv 函数监控命令接收情况。当上节提到的 g_uartCmdRecv.cmdReady 变量被置为 1 时，调用 CmdProcess 函数进行命令解析和处理。

CmdProcess 实现一个解析状态机用于校验命令数据包格式。只有收到命令包格式正确时，才会将命令包交由 CmdExec 函数处理。

8.4 波形参数配置

用户设计算法时，需根据算法需求设计符合条件的波形参数。一般按以下步骤进行：

1）选择合适的默认配置：

根据所购买的套件类型，在毫米波传感器的默认配置中选择对应模组：



图 8-11 传感器默认配置选择界面

点击“生成”按钮后生成对应模组的默认配置，基于此配置进行后续波形参数设置。

2）设计波形参数

用户可根据应用的实际情境，如检测频率，检测范围，目标能量，信噪比等参数，设计符合要求的波形参数。波形参数可参考章节：3 寄存器配置界面，如何配置 Chirp 时间及采样信息。

3）设置数据输出接口

- 传感器芯片数据输出接口：

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

毫米波传感器支持 SPI/UART/IO 等多种方式输出数据。ONELAB 硬件模组使用 SPI 方式与 MCU 进行数据传输。不同的模组因 MCU 差异，SPI 时钟频率也不尽相同。此处使用模组默认配置即可。



图 8-12 传感器芯片数据输出配置

波形配置时，需注意芯片的 SPI 单路瞬时速度不要超过时钟频率。有关数据输出速度的计算方法，可参考 如何计算实际输出的数据。

4）保存寄存器配置：

调整好波形后，可点击“生成固件配置”按钮，生成固件寄存器文件格式。参考《ONELAB 开发套件用户手册》“固件使用说明”章节将此文件转换为固件中的寄存器数组。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

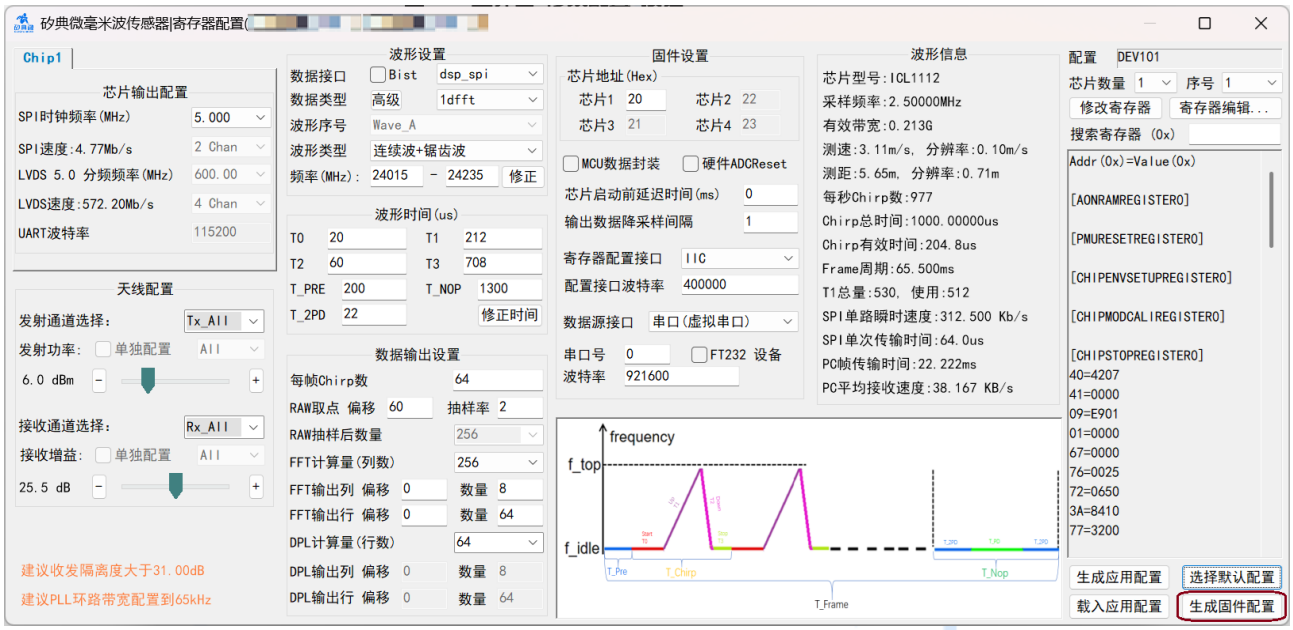


图 8-13 生成固件配置

9 版本历史

版本	时间	变更内容
1.0	2025/12/2	初版草稿。
1.1	2025/12/24	增加自定义硬件设计方案说明（第 8 章）
1.2	2026/03/05	优化部分截图，文字描述；增加第三方版权说明
1.3	2026/08/03	增加 ICL1240 相关模组配置，增加对应工具操作说明。
1.4	2026/09/24	修改图 2-1，图 6-6，图 7-1，图 7-2。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。

重要声明

矽典微“按原样”提供技术和可靠性数据（包括数据表）、设计资源（包括参考设计）、应用或其他设计建议、网络工具、安全信息和其他资源（以下简称“这些资源”），不保证没有瑕疵且不做出任何明示或者暗示担保，包括但不限于对适应性、某特定用途方面的适用性或不侵犯任何第三方知识产权的明示或者暗示担保。并特别声明不对包括但不限于产生于该应用或者使用任何本公司产品与电路造成的任何必然或偶然的损失承担责任。

矽典微保留对本文档发布的信息（包括但不限于指标和产品描述）和所涉及的任何本公司产品变更并恕不另行通知的权利，本文件自动取代并替换之前版本的相同文件编号文件所提供的所有信息。

这些资源可供使用矽典微产品进行设计的熟练开发人员使用。您将自行承担以下全部责任：(1) 针对您的应用选择合适的矽典微产品，(2) 全生命周期中设计、验证、运行您的应用和产品，(3) 确保您的应用满足所有相应标准，规范和法律，以及任何其他功能安全、信息安全、监管或其他要求。

矽典微授权您仅可将这些资源用于研发本资源所述的矽典微产品的应用。未经矽典微许可，任何单位和个人不得擅自摘抄、复制这些资源的部分或全部，并不得以任何形式传播。您无权使用任何其他矽典微知识产权或任何第三方知识产权。您应全额赔偿因在这些资源的使用中对矽典微及其代表造成的任何索赔、损害、成本、损失和债务，矽典微对此概不负责。

矽典微提供的产品受矽典微的销售条款或者矽典微产品随附的其他适用条款的约束。矽典微提供这些资源并不会扩展或以其他方式更改矽典微针对矽典微产品发布的适用的担保或担保免责声明。

版权文件

本文件及其附件版权为矽典微所有，未经授权，不得复制、分发或用于其他商业用途。